SpeedTower

Preproduction Specification

Group 7 | CS 4361.002

Prepared for Adrian Alvarez, Alexandru Istrate and Omar Sabry

Version 0.2 | October 13, 2026

SpeedTower is an endless, seeded voxel castle platformer with circumferential movement, independently rotating tower rings and stored or environmental boosts. This specification defines its initial gameplay, system contracts, graphics pipeline and implementation milestones.

The recommended design uses a custom cylindrical 2.5D controller with Three.js presentation and a generation worker. Movement tuning and performance targets require prototype validation. The package supplies editable source documents and diagrams for the team to maintain as development proceeds.

Document contents

- SpeedTower Game Design Brief
- SpeedTower Requirements and Traceability
- Proposed architecture and technical contracts
- World Generation and Streaming Specification
- Environments and Boost Catalogue
- Proposed design diagrams
- Implementation Roadmap and Validation
- Sources and Design Decisions

SpeedTower Game Design Brief

Purpose and player experience

SpeedTower is a local, single-player browser platformer about improving a route through practice. Its castle extends upward indefinitely. A seed creates a repeatable course of voxel platforms on independently rotating rings. Players climb, time moving surfaces, recover falls, and spend boosts to improve their route.

The screen-centered side view has two simulated movement axes: distance around the tower's circumference and height. The character remains at a fixed radius and apparent screen depth: a 2.5D model rendered with Three.js.

The first version includes six themes, procedural sections, three boosts, local records, and seed-based retry. Multiplayer, global leaderboards, input replay, checkpoints, and a finish line are excluded. This document specifies intended behavior, not an implemented or tested game.

Core loop and controls

Players enter or request a seed, begin on the safe base, and climb increasingly demanding sections within validated movement limits. The HUD shows active time, current/maximum height, Q/E contents, and latest split. Death offers a summary, same-course retry, or new seed.

Input	Behavior
A / Left arrow	Move left around the tower
D / Right arrow	Move right around the tower
Space	Jump; perform a wall jump when eligible
Q	Attempt the boost in the first slot
E	Attempt the boost in the second slot
Escape	Pause or resume

Opposing horizontal inputs cancel. Boosts activate on press edges; simultaneous presses resolve Q before E. Menus show controls, seed, course version, and timing status. Hazard/boost feedback uses silhouettes and symbols alongside color.

Movement, camera, and rotating surfaces

Run the controller at a fixed simulation step. Integrate ready sections and check required terrain at the tick boundary, then snapshot input, evaluate rings, expire boosts, process Q/E, resolve movement/contact, apply world pickups/pads, update metrics, and schedule streaming. Pads applied after collision affect the next tick. Use tangential distance as the horizontal coordinate and derive the render angle from the tower radius. Periodic collision handles crossing the circumference seam without a visible discontinuity. The camera follows the player's angle and height to maintain the centered side view, with a readable view of surfaces immediately above and below.

Each ring has its own seeded phase and angular velocity. Its phase is evaluated from simulation time, so unloaded rings return to the correct position. While grounded, the character is carried by its supporting ring in addition to controlled movement. On jumping or walking off an edge, inherit that ring's tangential velocity once. A landing on another ring establishes the new support; it must not add the previous ring's velocity again.

Swept collision considers player and platform motion, preventing tunneling during dashes, opposing-ring landings, and seam crossings. Shared gameplay geometry produces collision and voxel surfaces; each rotating ring remains a separate mesh.

Wall slide slows descent against an eligible solid wall. Wall jump pushes the player upward and away from the contacted wall. Coyote time and a short jump buffer improve responsiveness, but route validation must not require either grace mechanism. Visual effects may communicate speed without obscuring the next landing surface.

Candidate tuning	Initial value
Tower radius / voxel edge	8 units / 1 unit
Player width / height	0.7 / 1.6 units
Controlled move speed	6 units/s
Gravity / fall speed cap	24 units/s² / 24 units/s
Jump launch / air acceleration	10 units/s / 18 units/s²
Fixed step	1/120 s
Coyote time / jump buffer	0.10 s / 0.12 s
Wall slide descent cap	3 units/s
Wall jump upward / outward speed	10 / 7 units/s
Dash speed / duration	18 units/s / 0.16 s
High-jump launch	14 units/s
Glide descent cap / duration	3 units/s / 2 s

These unvalidated prototype defaults are configurable game units. Tune the controller first, then derive a conservative route-validation envelope. Required jumps remain feasible without boosts.

Boost inventory and pads

Q and E each hold one consumable charge. A world pickup fills Q first, then E. Duplicate boost types are allowed. When both slots are occupied, the pickup remains available in the world. A charge is removed only after its effect successfully activates.

Type	Effect and successful activation
Dash	Tangential burst in the current input direction, or facing direction without input; rejects activation while dash is active
High jump	Sets upward velocity to the configured launch value only when that increases current upward velocity; works grounded, at a wall, or airborne
Glide	Caps downward velocity while airborne; rejects activation when grounded or already gliding; ends on landing or expiry

Different types can combine: for example, glide can control descent during a dash. Repeated activation never multiplies an existing effect. An invalid activation preserves the inventory charge and provides brief HUD feedback. Two simultaneous high-jump charges therefore spend only the first if it has already established the full launch velocity.

Pads apply the same effect definitions immediately on entry. Each pad latches after an entry attempt, including an unsuccessful attempt, and cannot retrigger while the character remains inside it. A successful activation starts a 0.5-second cooldown. Rearming requires both exit and cooldown completion; an unsuccessful activation requires exit but starts no cooldown. Reentering too early remains ineligible until another exit and entry. This prevents continuous overlap from producing repeated launches or repeated invalid attempts.

Falling, hazards, and course structure

Ordinary falls remain recoverable: lower sections stream back into memory, including when the player falls a large distance. The starting base catches nonhazard falls. There is no arbitrary death plane or checkpoint reset. Lava and clearly designated lethal hazards end the run immediately; retry rebuilds the same course from the base and clears that run's collected-item history.

Build each section around a baseline route, then add decorative voxels and optional boost shortcuts. Waiting surfaces support rotating-ring timing challenges. Difficulty increases by combining validated patterns and bounded parameters, not by increasing gaps indefinitely. A deterministic safe template replaces a candidate that fails route validation.

Environment	Visual direction and challenge emphasis
Stone Ruins	Warm stone, broken battlements, introductory ledges and ring timing
Overgrown Keep	Moss and vines, wall transitions and narrow readable footholds
Frost Citadel	Cool crystalline masonry, deliberate timing and exposed platforms
Volcanic Forge	Dark metal and orange light, explicitly marked lava hazards
Arcane Tower	Violet light and runic surfaces, counter-rotating ring combinations
Storm Battlements	Wind-swept silhouettes, exposed climbs and restrained atmosphere

Themes reuse the movement model. Visual frost, vines, or wind do not silently introduce friction or force changes. Additional mechanics need explicit requirements and route validation before introduction.

Timing, records, and suspension

Track the greatest height reached and active elapsed time. Record each section's first upward boundary crossing once. Falling does not reduce the height watermark or replace a split. For local comparisons, higher maximum height ranks first; tied height compares the earlier first time that height was reached. Do not use the final death time as the tie-breaker.

Use a monotonic active clock with millisecond display precision. That presentation does not imply millisecond physics resolution. User pause, a hidden tab, missing required terrain, and a blocking storage operation suspend both simulation and timing. Initial preparation happens before the active clock begins. Any suspension during an active run marks its timing as interrupted; the summary makes that flag visible.

Compare local records only when course identity, movement rules, and timing flags match. Course identity includes seed and generator version, and binds the movement-rules version. Repeatable layouts are required; exact input replay and cross-machine physics identity are not. Store records and collected-item history locally. Distant collectible state survives section regeneration, while its persistent journal may grow during a long run. Active world geometry, buffers, and queued jobs must remain bounded.

SpeedTower Requirements and Traceability

Interpretation

These are future implementation targets, not completed test results. `FR` identifies behavior; `NFR` identifies quality constraints. `TC-01` through `TC-20` refer to the validation plan. Responsible systems own behavior; the roadmap assigns people.

The proposal establishes project/team context; the accepted plan establishes gameplay decisions. Changing candidate movement constants requires a rules-version change and route revalidation. The architecture defines the named systems; the validation plan defines test IDs.

Player use cases

ID	Player intent	Successful outcome
UC-01	Start a course	A chosen or generated seed starts on the safe base
UC-02	Climb rotating platforms	Predictable movement, jumps, and ring carry permit progress
UC-03	Recover a fall	Lower terrain returns and permits a continued climb
UC-04	Collect and spend boosts	Q/E show charges and successful effects consume them
UC-05	Use an environmental pad	Entry applies an eligible effect with readable feedback
UC-06	Pause and return	Simulation and active timing resume together
UC-07	Review and retry	Summary reports height, splits, flags, and same-seed retry
UC-08	Improve a local record	Comparable course and timing categories show personal progress

Functional requirements

ID	Requirement	Responsible system	Acceptance and test
FR-01	Start with a chosen seed or generate a seed; display course identity	GameSession	Same identity is visible at start and summary; TC-12
FR-02	Support A/D, arrows, Space, Q/E, Escape and centered side view	InputController; CameraRig	Controls and fixed apparent depth match the brief; TC-01
FR-03	Simulate fixed-radius tangential and vertical movement with fixed steps and swept collision	PlayerController; CollisionWorld	No tunneling or seam discontinuity at maximum configured speed; TC-05
FR-04	Carry grounded players with their supporting rotating ring	PlayerController; RingSystem	Idle player follows the support ring displacement; TC-02
FR-05	Inherit departure velocity once and establish new support on landing	PlayerController; CollisionWorld	Jump, walk-off, and opposing-ring landing add no duplicate carry; TC-03
FR-06	Provide wall slide and wall jump	PlayerController	Slide cap and upward/outward launch match configuration; TC-04
FR-07	Keep a baseline route traversable without boosts	SectionGenerator	Validated required transitions fit the conservative movement envelope; TC-13

ID	Requirement	Responsible system	Acceptance and test
FR-08	Generate sections independently from seed, index, and version	SectionGenerator	Request order and regeneration produce equivalent route/decoration data; TC-12
FR-09	Validate candidates and use a deterministic safe fallback	SectionGenerator	Failed candidates produce the approved connected fallback; TC-13
FR-10	Provide six themes with readable, explicit hazards	SectionGenerator; CollisionWorld	Theme catalogue is represented; cosmetic effects add no hidden forces; TC-14
FR-11	Fill Q then E with one charge each; allow duplicates	BoostInventory	Full inventory leaves pickup unspent in world; TC-09
FR-12	Activate boosts on press edges, Q before E, consuming only on success	BoostSystem	Held keys do not repeat; rejected effects retain charges; TC-09
FR-13	Dash tangentially using input or facing direction	BoostSystem; PlayerController	Collision applies during burst; active dash rejects repeats; TC-06
FR-14	High jump increases upward velocity to configured launch speed	BoostSystem; PlayerController	Ground, wall, and air launches work; no increase rejects consumption; TC-07
FR-15	Glide caps descent while airborne and ends on landing/expiry	BoostSystem; PlayerController	Grounded/active rejection and cross-type combination behave correctly; TC-08
FR-16	Pads trigger on entry, latch through overlap, and rearm after exit/cooldown	BoostSystem	Successful activation starts 0.5 s cooldown; invalid overlap never repeats; TC-10
FR-17	Recover ordinary falls; terminate on lethal hazards; retry from base	GameSession; CollisionWorld	Base catches safe falls; retry clears attempt state without checkpoints; TC-11
FR-18	Stream terrain above and below the player, including long falls	ChunkStreamer	Required terrain is available or run suspends before entry; TC-15
FR-19	Preserve spent pickups across section unloading and regeneration	RunStateStore	Collected IDs remain absent until a fresh retry; TC-15
FR-20	Reject stale worker results and activate complete sections atomically	ChunkStreamer; GenerationWorker	Canceled/old-run results never alter active collision or visuals; TC-16
FR-21	Suspend simulation and active clock for pause, hidden tab, loading, or storage	GameSession	All suspension paths resume together and flag active runs; TC-17
FR-22	Track maximum height and monotonic active time	RunTracker	Falls preserve height; suspended time is excluded; TC-18
FR-23	Record each upward section split once	RunTracker	Repeated ascent does not overwrite first-arrival splits; TC-18
FR-24	Store summaries and compare matching course/rules/timing flags locally	RunStateStore; RunTracker	Rank height first, then earlier first reach; interrupted status visible; TC-18
FR-25	Rebase large render coordinates while retaining logical heights	RenderSystem; CollisionWorld; CameraRig	Rebase preserves collision, ring phase, timing, and splits; TC-19

Quality requirements

ID	Constraint	Responsible system	Acceptance and test
NFR-01	Target 60 FPS on documented desktop hardware	RenderSystem; ChunkStreamer	Report frame-time percentiles during 30-minute traversal; TC-20
NFR-02	Bound runtime world, collision, and buffer memory	ChunkStreamer	Runtime plateaus under sustained travel; growing disk journal reported separately; TC-15, TC-20
NFR-03	Bound worker concurrency, job queue, and uploads	GenerationWorker; RenderSystem	Configured limits hold under rapid ascent/fall; no unbounded backlog; TC-16, TC-20
NFR-04	Reproduce layouts using versioned generation inputs	SectionGenerator	Seed/order/version checks pass; input replay is not promised; TC-12
NFR-05	Dispose unloaded GPU resources and reusable buffers correctly	RenderSystem; ChunkStreamer	Repeated unload/reload causes no monotonic resource growth; TC-20
NFR-06	Make hazards, boosts, and landing surfaces visually readable	SectionGenerator; HUD	Symbols/silhouettes supplement color; tested at target viewport; TC-01, TC-14
NFR-07	Maintain responsive, stable movement	InputController; PlayerController	Press edges, grace windows, and sweeps remain stable across render rates; TC-01, TC-05
NFR-08	Preserve consistent local state on storage failure	RunStateStore; GameSession	Blocking failures suspend safely with retry feedback; no silent collectible reset; TC-15, TC-17
NFR-09	Keep documents, diagrams, requirements, and reports traceable	Documentation pipeline	Every requirement has an owner/acceptance; rendered reports agree with sources; artifact review
NFR-10	Detect unavailable browser capabilities before play	GameSession	WebGL2/worker/storage checks provide a useful failure message; TC-17

Traceability and acceptance gates

Use case	Principal requirements	Planned scenarios
UC-01	FR-01, FR-08–10	TC-12–14
UC-02	FR-02–07, FR-13–15	TC-01–08, TC-13
UC-03	FR-17–20, FR-25	TC-11, TC-15–16, TC-19
UC-04	FR-11–15, FR-19	TC-06–09, TC-15
UC-05	FR-16	TC-10
UC-06	FR-21	TC-17
UC-07	FR-17, FR-22–24	TC-11, TC-18
UC-08	FR-01, FR-22–24	TC-12, TC-18

Validate movement before tuning generation. Then check seed batches, section boundaries, safe fallbacks, bidirectional streaming, collectible persistence, stale worker replies, and rebasing.

The performance gate records hardware/browser, resolution, settings, seed, traversal, and resources; 60 FPS remains unverified. Inspect every exported page for diagram errors, overflow, consistency, and traceability. Later behavior changes update requirements, diagrams, and tests together.

Proposed architecture and technical contracts

Status: pre-production proposal. These interfaces describe intended modules; no game implementation or performance result exists yet.

Architecture alternatives

Option	Structure	Tradeoff
A: single-thread custom 2.5D	Three.js, player/collision systems, generation and meshing on the main thread	Fastest prototype; generation can interrupt movement and rendering.
B: modular 2.5D with generation worker	Main-thread gameplay and rendering; one module worker builds deterministic sections and meshes	**Recommended.** Clear team boundaries and bounded work queues, with manageable message contracts.
C: Three.js and Rapier 3D	Constrain a 3D character to a cylinder; kinematic rotating ring bodies	Useful for future radial movement; requires more contact, constraint, and synchronization work.

The confirmed fixed apparent depth makes B appropriate: authoritative movement has two coordinates, unwrapped arc distance `x` and logical height. Three.js presents the result in 3D. Rapier's controller supports translational movement and sliding, but explicitly excludes rotational character movement; it would not remove the need to define circular movement and rotating contacts. Rapier controller

Stack, ownership, and frame flow

Use TypeScript, Vite, direct Three.js `WebGLRenderer`/WebGL2, an HTML/CSS HUD, a module Web Worker, and IndexedDB. Pin dependency versions in the future lockfile. Use Vitest for pure generation/state tests and Playwright for browser scenarios when implementation begins. Three.js documents WebGL2 as the renderer baseline. WebGLRenderer

`GameSession` coordinates `SimulationClock`, `InputController`, `PlayerController`, `CollisionWorld`, `RingSystem`, `BoostSystem`, `BoostInventory`, `ChunkStreamer`, `RenderSystem`, `CameraRig`, `HUD`, `RunStateStore`, and `RunTracker`. It passes explicit state and commands; the renderer never owns gameplay decisions.

At each 120 Hz tick, integrate ready chunks and verify required terrain; snapshot input; advance ring phases; expire effects; process Q then E; sweep player movement; process world triggers; update run metrics and streaming requests. Pad effects begin at the next movement step. Rendering interpolates between completed states. Never skip gameplay ticks to hide a slow frame; cap catch-up work and suspend with an interruption flag when necessary.

Ground contact carries the player with its ring while controls add relative tangential movement. `vx` stores world tangential speed: grounded updates compute `vx = relativeSpeed + supportRingSpeed`; departure retains that `vx` without adding support speed again. Landing subtracts the new support speed from incoming world `vx` before friction/control adjusts relative movement. Grounded dash sets signed self speed to 18; airborne dash replaces world `vx` with signed 18. Both preserve vertical velocity. Sweeps use player motion relative to each ring. Boosts are orthogonal to movement mode.

`RingSystem` derives each phase from its seed-defined initial phase and simulation tick. `RenderSystem` rotates each ring by its world phase relative to the player's angle `x / radius`; `CameraRig` keeps the exterior player centered at fixed apparent depth. Use the same platform descriptions for geometry and collision. Wrap angles locally, while keeping `x` unwrapped.

Proposed interfaces

These are documentation snippets, not source files. Validate `seed` as a canonical uint32 before creating `RunConfig`. A course identity is `(seed, generatorVersion, rulesVersion)`; `runId` identifies one attempt and is excluded from course identity.

```
type MovementMode = "grounded" | "airborne" | "wallSliding" | "dead";
type BoostKind = "dash" | "highJump" | "glide";
```

```
type LogicalHeight = { sectionIndex: number; localHeight: number };
type RunConfig = {
  seed: number; generatorVersion: string; rulesVersion: string; runId: string;
};
type SectionKey = Omit<RunConfig, "runId"> & { sectionIndex: number };
interface PlayerState {
  x: number; logicalHeight: LogicalHeight; vx: number; vy: number;
  supportRingId: string | null; facing: -1 | 1; movementMode: MovementMode;
}
interface RingSpec {
  id: string; localHeight: number; phase0: number; angularSpeed: number;
}
interface PlatformSpec {
  id: string; ringId: string;
  angleIntervals: readonly { start: number; end: number }[];
  bottom: number; top: number;
}
interface BoostDefinition {
  id: string; kind: BoostKind; duration: number; strength: number;
}
interface SectionTemplate {
  id: string; ringCountRange: readonly [number, number];
  routeSockets: readonly RouteSocket[]; maxAngularSpeed: number;
  reservedHeadroom: number; allowsLava: boolean; routeConstraints: RouteConstraints;
}
interface RouteSocket {
  kind: "launch" | "landing" | "wall";
  localHeight: number; arcOffset: number; arcWidth: number;
}
interface BoundaryProfile {
  boundaryIndex: number; ring: RingSpec; platform: PlatformSpec;
  reservedHeadroom: number;
}
interface RouteConstraints {
  maxRise: number; maxArcGap: number; minLandingWidth: number;
}
interface WorldEntitySpec {
  id: string; kind: "pickup" | "pad" | "hazard"; platformId: string;
  angleIntervals: PlatformSpec["angleIntervals"];
  bottom: number; top: number; boostId?: string;
  hazardKind?: "lava" | "spike" | "thorn" | "crystal" | "rune" | "conductor";
}
interface RingMeshBuffers {
  ringId: string; positions: Float32Array; normals: Float32Array;
  colors: Float32Array; indices: Uint32Array;
}
interface ChunkBuildRequest {
  type: "build"; runId: string; requestId: number; key: SectionKey;
}
interface ChunkBuildResult {
  type: "built"; runId: string; requestId: number; key: SectionKey;
  rings: readonly RingSpec[]; platforms: readonly PlatformSpec[];
  entities: readonly WorldEntitySpec[]; meshes: readonly RingMeshBuffers[];
}
interface ChunkBuildError {
  type: "error"; runId: string; requestId: number; key: SectionKey;
  message: string;
}
interface RunRecord {
  course: Omit<RunConfig, "runId">; maxHeight: LogicalHeight;
  timeAtMaxHeightMs: number; activeTimeMs: number;
  interrupted: boolean; boostsUsed: number;
}
interface SectionSplit { sectionIndex: number; firstCrossingMs: number }
interface RunStateStore {
  loadSpent(runId: string, key: SectionKey): Promise<ReadonlySet<string>>;
  persistSpent(runId: string, key: SectionKey, ids: readonly string[]): Promise<void>;
```

```
  flush(runId: string): Promise<void>;
  persistSplit(runId: string, split: SectionSplit): Promise<void>;
  loadSplits(runId: string, fromSection: number, limit: number): Promise<readonly
    SectionSplit[]>;
  getRecord(courseId: string, interrupted: boolean): Promise<RunRecord | undefined>;
  saveRecord(record: RunRecord): Promise<void>;
}
```

`persistSplit` writes only if that section has no prior split. `loadSplits` pages at most 64 entries. Record storage keys include course identity and interruption category.

Distances/speeds use game units and units/second; angles/speeds use radians and radians/second; durations use seconds. Height normalizes to a nonnegative integer section index and local height in `[0, 32)`. Vertical collision bounds are section-local; mesh positions and angular intervals are ring-local. Add ring phase for world intervals. Phase time is `tickIndex / 120` seconds. Split intervals at the seam with `0 <= start < end <= 2π`; full-circle intervals are allowed. `strength` means self dash speed, high-jump launch speed, or glide fall cap. Hazard styles share one lethal rule.

`GenerationWorker` returns transferable typed-array buffers and plain specifications, never Three.js objects. Transferring a buffer detaches it from its sender, so the worker must not reuse transferred arrays. Transferable objects

Determinism, generation, and streaming

`SectionGenerator` builds a baseline route, validates a candidate against movement envelopes and sampled ring phases, then decorates it. Try at most eight candidates; use a known safe deterministic fallback. `boundaryProfile(n)` defines the stationary broad landing at height `32 * n`: section n owns its lower boundary and targets section `n + 1`'s lower landing. Both neighbours use the same keyed profile. Destination terrain must be resident before crossing. This is validation evidence, not a mathematical proof of reachability.

Key each random stream with UTF-8 bytes of `seed|generatorVersion|rulesVersion|sectionIndex|streamTag|candidateId`. Numbers are canonical unsigned decimal strings; version/tag identifiers contain only ASCII letters, digits, `.`, `_`, and `-`. Tags are `boundary`, `route`, `ring`, `platform`, `boost`, `decoration`, `hazard`, and `fallback`; boundary profiles use candidate `0`. Hash with uint32 FNV-1a: start `2166136261`, then for each byte use `Math.imul(hash ^ byte, 16777619) >>> 0`. Feed that hash to this Mulberry32 contract:

```
function mulberry32(state: number): () => number {
  return () => {
    let t = state = (state + 0x6D2B79F5) >>> 0;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

Fixed traversal order and separate streams prevent decoration, generation order, and worker timing from changing routes. Same seed reproduces specifications and initial phases. Simulation replay requires additional guarantees and is outside the initial deliverable. Rapier determinism caveats

`VoxelMesher` removes internal faces and greedily merges compatible faces within each ring; independently rotating rings never share face-culling or merged geometry. Repeated props may use instancing. One mesh per voxel is unsuitable. Three.js voxel geometry, InstancedMesh

`ChunkStreamer` ordinarily retains six sections: current, two below, and three above; adaptive downward prefetch can raise the hard cap to eight. Limit queued builds and ready results to two each. Reject stale run/request/version results. Atomically activate geometry, collision, entities, and loaded spent flags at a tick boundary. On return, regenerated rings use the current tick rather than restarting their phase.

Mark collected pickups spent in resident state immediately, then journal stable IDs in IndexedDB; flush before eviction. Reusable pads need resident entry/cooldown state. Bound caches and pending writes; disk can grow during a long run. Death retains metrics/splits until summary storage completes; a new attempt resets its journal. Storage failure suspends with storage-retry/restart choices. Dispose ring geometry on eviction, retaining shared materials until unused. Three.js resource disposal

Graphics pipeline and shader suite

Render opaque voxel rings first, followed by pickups, pads, the avatar and atmospheric scenery. The HTML HUD is independent of post-processing. Keep materials and texture atlases shared across resident sections; update ring transforms rather than remeshing rotating geometry.

Pass or asset	Initial design	Validation
Voxel surface shader	GLSL vertex transforms and normal transforms; fragment lighting combines a diffuse response, ambient light, vertex-colour AO and bounded torch attenuation. Greedy merges preserve material and corner-AO compatibility.	Ring rotation changes lighting without moving collision differently; AO never erases a landing edge.
Torch and shadow lighting	At most four nearby torch lights; one directional shadow map, initially 1024 by 1024, covering the active player region. Use soft filtered shadows and emissive torch/lava surfaces.	Toggle shadows and torch count independently; measure passes and frame cost before increasing quality.
Parallax background	A background shader uses layered scenery and camera height/angle offsets to suggest distant castle surroundings. It does not supply collision.	Long climbs and origin rebasing preserve smooth offsets without discontinuities.
Speed post-processing	A scene render target feeds a restrained vignette and directional motion-cue pass during boosts. Default to no full-screen motion blur so upcoming platform edges remain legible.	Disable the pass for a cost comparison; verify HUD text and hazard silhouettes remain clear.

The lighting limits and shadow resolution are prototype settings. Document shader uniforms, vertex inputs and per-pass costs in the M4 shader suite. If a quality setting misses the benchmark budget, reduce shadow coverage, light count or post-processing before lowering gameplay simulation frequency. These are proposed graphics designs, not compiled or measured shaders.

Runtime budgets and failure behavior

Provisional limits: radius `8`, section height `32`, tangential speed `26`, downward speed `24`, upward speed `14`, and ring tangential speed at most `4`. Rebase render coordinates every `512` height units without changing logical height or seeds. Benchmark at 1080p, DPR 1, on documented hardware; tune these values together if streaming or collision budgets fail.

Pause, hidden-tab, missing-terrain, and storage suspensions freeze simulation, rings, effects, and active timing. Set `RunRecord.interrupted` when an active run stops timing. `RunTracker` uses a monotonic active wall clock with milliseconds displayed; fixed-step resolution does not establish submillisecond accuracy. Track highest height, its first-reaching time, elapsed active time, and boosts used. Keep the latest split resident and journal historical first-crossing splits in IndexedDB. No checkpoint or trailing death plane exists; lower platforms remain recoverable by regeneration. Lava templates and the base boundary supply explicit hazards/bounds.

Acceptance requires bidirectional revisits without pickup respawns, bounded resident resources/queues, high-speed swept contacts, matching mesh/collision phases, and a loading suspension before entering missing terrain. Performance targets require measurement; none have been tested yet.

World Generation and Streaming Specification

SpeedTower generates an endless upward castle from a numeric seed. This contract implements FR-07–10, FR-18–20, and FR-25 from Requirements. No generator or reachability test exists yet. Numerical settings are prototype candidates for Milestones 1–3.

Coordinate system and section boundaries

Use a fixed tower radius of **8 units**, a **1-unit voxel**, and **32-unit vertical sections**. Section indices are nonnegative integers; section 0 contains the base floor. The authoritative player location is section index plus local height. Horizontal `x` is unwrapped distance along the circumference, whose length is `2π × 8`; rendering converts it to an angle. Collision operates on circumference distance and height, while the rendered tower provides depth.

Platforms, walls, hazards, and boosts belong to an independently rotating ring or stationary group. Shared gameplay geometry produces collision and visible voxels; decoration must not conceal landings or create unrepresented obstacles.

A pure `boundaryProfile(n)` defines a shared stationary landing at logical height `32 × n`. Its arc centre comes from the boundary random stream, and its initial landing width is 4 units. Boundary 0 is a safe base floor spanning the full circumference. `generate(n)` uses boundary profiles n and `n + 1`, owns the lower landing, and reserves clearance around both. Its exit targets the successor's lower landing. This gives independently generated neighbours compatible entrances and exits without requiring worker order or predecessor state. The destination section must be resident before the player can cross into it.

Seed and identity contract

Accept only canonical unsigned decimal seeds from **0 through 4294967295**. Course identity combines seed, generator version, and movement-rules version. A tuning change that affects reachability or rotation changes the rules version; a layout algorithm or template change changes the generator version.

Use the exact FNV-1a keyed-stream and Mulberry32 algorithms defined in Architecture and Tech Stack. Keys include seed, versions, section index, stream tag, and candidate identifier. Use independent `boundary`, `route`, `ring`, `platform`, `boost`, `decoration`, `hazard`, and `fallback` tags. Route changes cannot consume decoration draws, and asynchronous job order cannot alter a section. Never use `Math.random()` or wall-clock time to generate course content.

`generate(sectionIndex)` is pure for a course identity and versioned template catalogue. Gameplay descriptors and mesh inputs are independent of request order. Stable entity IDs contain course identity, section index, and local ID; collectible state adds `runId`. Canonical descriptor hashes support TC-12.

Route-first generation

The worker performs these stages in order:

1. Read entrance/exit boundary profiles and the saturated difficulty value. Select the section theme and a compatible template family.
2. Build a baseline route using platforms, climbable walls, rotating rings, and safe waiting surfaces. Add optional boost shortcuts and lower recovery platforms.
3. Validate with the actual controller, moving-platform collision, and inherited ring velocity; exclude jump-buffer/coyote-time assistance.
4. Add hazards only outside reserved route and waiting clearances; validate the affected route again.
5. Add decorative voxels, then extract collision descriptors and independently mesh each rigid ring group.
6. Return immutable descriptors and transferable mesh buffers for staged main-thread activation.

`SectionTemplate` defines entrance/exit profiles, route sockets, allowed ring speeds, clearances, and optional placements. `RingSpec`, `PlatformSpec`, hazard descriptors, and boost placements are documented with the worker contract in Architecture and Tech Stack. Templates should originate from a small hand-tested catalogue, then receive offline validation before wider procedural combinations are enabled.

Every mandatory transition must work with baseline movement and without consumable boosts. A boost shortcut may be faster or save a failed jump, but must never be the sole route upward. Provide waiting surfaces where ring alignment is required and recovery platforms below ordinary gaps. Lava is an explicitly lethal exception, with readable warnings and a baseline-safe crossing.

Try candidate identifiers **0–7** in order. Reject a candidate when a required transition fails the validator. After eight failures, use a deterministic stationary safe-connector template matched to the two boundary profiles. The fallback has no required boosts or route hazards. Validate it too; if it fails, report a generation error and suspend at the ready frontier instead of admitting an invalid section.

Validate **16 representative rotation phases** with legal waits, jumps, and wall movement. Sampling is not a proof covering every timing state. Report failed transitions and sampled states. TC-13 proposes **100 seeds × 100 sections**, including fallback and boundary compatibility; hand traversal remains required.

Rotation and bounded difficulty

Evaluate each ring's phase from its seeded initial phase and fixed simulation tick: `phase(t) = initialPhase + angularSpeed × t`, where `t = tickIndex / 120` seconds, normalized only when needed. Regenerating a ring evaluates its current phase, rather than restarting its animation. Collision samples the same phase as rendering. Grounded players inherit ring displacement; jump or step-off inherits tangential velocity once.

Increase route complexity gradually, then saturate after section **40**. Scale only within the catalogue's validated gap, wall-height, waiting-time, and rotation limits. Do not keep increasing jump distances with height. The absolute ring speed cap is **0.5 rad/s**, or **4 units/s** tangentially at radius 8. The controller's provisional maximum total horizontal speed is 26 units/s, upward speed 14 units/s, and downward speed 24 units/s. Generation uses those caps for streaming safety, and baseline jump parameters for route validation.

Streaming, regeneration, and persistence

The normal resident window contains the current section, **two below**, and **three above**, clamped at section 0. Keep at most **eight resident sections**, allowing two additional direction-dependent prefetch sections. Limit worker generation to **two jobs** and completed upload payloads to a **two-result queue**. Budget their buffers separately from resident GPU geometry; bounded section counts alone do not guarantee bounded byte use.

Estimate prefetch distance from the relevant velocity cap multiplied by measured generation, transfer, and upload latency, plus a safety margin. The initial worker generation P95 target is **150 ms per section**. Measure upload latency as well; adjust section complexity and prefetch within the cap. These are targets, not established performance. Crossing a missing frontier suspends simulation ticks, rings, and active run timing with a visible loading state. A suspension after the run starts marks it interrupted (TC-17).

Stage a result until all ring meshes, collision data, and collectible journal entries are ready, then activate it atomically between ticks. Include a run token and course versions in requests/results; discard stale results after retry or seed changes (TC-16). Unload geometry, collision, and render resources only outside the required window. Ordinary falls can descend through regenerated sections indefinitely, down to the safe base; crossing a streaming boundary is never a death condition.

Use IndexedDB for a spent-pickup run journal. Load a section's journal before activation, and persist dirty collection entries before evicting its state. Keep only resident entries and a bounded pending-write queue in memory. Disk history can grow with the number of collected items during a long run. A failed journal operation pauses the run with retry/restart actions; never silently restore spent pickups. Retry creates a new `runId` and empty journal while retaining course identity (TC-15).

Rebase local vertical coordinates every **512 units**. Preserve authoritative section/local height, maximum height, timer, splits, and entity identities. Horizontal distance remains unwrapped double precision; normalize derived angles for rendering. Rebase tests must cover ascent and descent (TC-19).

Voxel and verification budgets

Cull hidden faces only within the same rigid moving group. Merge faces only when normal, material, and ambient-occlusion attributes match; independently rotating neighbours can expose faces later. Keep collision descriptors independent of greedy mesh topology. Bound lights, shadows, draw calls, vertices, and allocated bytes through measured graphics budgets.

TC-14 checks theme readability; TC-20 specifies a 30-minute ascent/descent soak targeting 60 FPS on recorded hardware. Measure frame-time percentiles, generation/upload latency, draw calls, vertices, section counts, JS/buffer/GPU resource estimates, and journal growth. Reject missing-terrain collisions, unbounded resident-resource growth, or pickup duplication. See Roadmap and Validation.

Environments and Boost Catalogue

Six environments reuse one movement model and hand-tested challenge patterns (FR-10); boosts implement FR-11–16. See Requirements. Colours and numbers are art and tuning candidates.

Environment direction

Use four-section theme blocks. For the ordered catalogue below, select `(seed % 6 + floor(sectionIndex / 4)) % 6`; repeat indefinitely. Themes transition at the shared stationary landing. Blend scenery/light colour without changing physics. Preserve readable landing tops, wall-slide surfaces, boosts, and lethal warnings.

Environment	Palette candidates	Visual and challenge direction
Stone Ruins	Stone `#777A80`, charcoal `#272A30`, torch `#FFBC63`	Broken crenellations, wide stone tops, and open sightlines teach ring carry, short gaps, and wall jumps.
Overgrown Keep	Moss `#527A4F`, stone `#777260`, leaf `#9ABA67`	Vines and arches frame staggered platforms and alternating rings; route surfaces stay free of foliage.
Frost Citadel	Ice `#B8E5F1`, slate `#4E6984`, accent `#E8FAFF`	Sharp crystal silhouettes and cool light surround longer waits and narrow, high-contrast landings.
Volcanic Forge	Basalt `#37313C`, lava `#FF643A`, metal `#A08A78`	Thick anvils, furnace vents, and lit crossings frame explicit lethal lava gaps.
Arcane Tower	Violet `#6650A7`, midnight `#272A51`, rune `#78E1DD`	Floating-looking voxel supports and rune strips frame alternating directions and wall-jump chains.
Storm Battlements	Blue-grey `#526A7A`, cloud `#B8C8D1`, amber `#FFD16B`	Large battlement silhouettes, cloud parallax, and visible ring edges support demanding timing sequences.

Candidate rotation ranges below are magnitudes in rad/s at full difficulty. A seeded direction belongs to each ring; ramps scale within the configured limits, and designated waiting platforms remain stationary. The global cap remains 0.5 rad/s.

Theme	Rotation range / pattern	Hazard and optional shortcut
Stone Ruins	0.10–0.18; simple same-direction pairs	Clearly marked spikes away from waiting zones; dash crosses a shorter optional loop.
Overgrown Keep	0.08–0.16; gentle opposite pairs	Visible thorn blocks use the standard lethal-hazard rule; high jump skips an arch ascent.
Frost Citadel	0.12–0.22; offset alignment windows	Marked spike crystals; glide recovers onto a lower ledge. Ice is cosmetic initially.
Volcanic Forge	0.16–0.28; interrupted platform arcs	Luminous lava surfaces kill on contact; dash offers an optional crossing alongside a baseline route.
Arcane Tower	0.18–0.35; alternating ring directions	Always-visible lethal rune blocks; high jump bypasses an optional wall chain.
Storm Battlements	0.22–0.40; mixed timing patterns	Marked lethal conductors; glide accesses a broad lower shortcut. Wind and lightning remain visual effects.

Use the same hazard-contact rule across themes, with distinct silhouettes. Slippery movement, gust forces, timed damage, and reduced visibility require separate rules and tests. Darkness must preserve landing contrast. Icons and shapes supplement boost/hazard colours (TC-14).

Shared boost definitions

Items and level pads invoke the same `BoostDefinition` and activation path. Q and E are independent one-item slots. A pickup fills Q first, then E; duplicate types are allowed. If both are occupied, leave the pickup present. Resolve simultaneous Q/E requests in Q-then-E order using the state resulting from the first activation. Consume a slot only after its effect successfully activates (TC-09).

Boost	Candidate effect	Activation rule and failure
Dash	Set self horizontal speed to 18 units/s for 0.16 s; preserve vertical velocity	Use held movement direction, otherwise facing. Respect swept collision and the 26-unit/s total cap. Reject while a dash is active.
High jump	Set vertical velocity to +14 units/s	Available grounded, wall-sliding, or airborne, including during descent. Succeed only if this increases current vertical velocity; otherwise retain the item.
Glide	Cap descent at 3 units/s for 2 s	Activate only airborne or wall-sliding. End on landing or expiry. Reject while already active or grounded.

While grounded, dash sets relative tangential speed to the signed 18-unit/s target, then adds the support ring speed to obtain world `vx`. An airborne dash replaces world `vx` with the signed 18-unit/s target; it replaces previously inherited tangential momentum and preserves vertical motion. Ordinary takeoff without a dash retains the already computed world velocity rather than adding ring speed twice.

When dash meets a wall, stop horizontal motion through the ordinary collision response; the effect timer continues to expiry. High jump detaches from its supporting surface and retains world tangential velocity under the normal departure rule. Glide clamps downward velocity only, so it does not cancel an upward jump. Effects advance on simulation ticks and freeze during suspension.

Different boost types can coexist. Apply the vertical impulse, then persistent effect constraints during each tick: high jump sets upward velocity, dash controls horizontal speed, and glide limits descent. Glide continues across a high jump until landing or expiry. A second high jump can succeed after vertical velocity drops below 14 units/s, but never stacks impulses above that value. Dash and glide do not multiply or extend an already active effect (TC-06–08).

Pads activate on trigger entry. Rearming requires complete exit and expiry of the **0.5-second simulation-time cooldown** started by success. Failed entry also requires exit before retry. Pads are reusable, consume no inventory, and follow input-before-contact ordering from the architecture. Only success starts cooldown (TC-10).

Placement, feedback, and balance

Required routes remain traversable with baseline movement. Put pickups on visible approach surfaces and pads on deliberately signposted shortcuts. Reserve enough space to see the destination before committing to dash; glide shortcuts need a viable lower landing. Do not place a required pad at an impossible-to-repeat timing window or use a pickup to compensate for a failed baseline route.

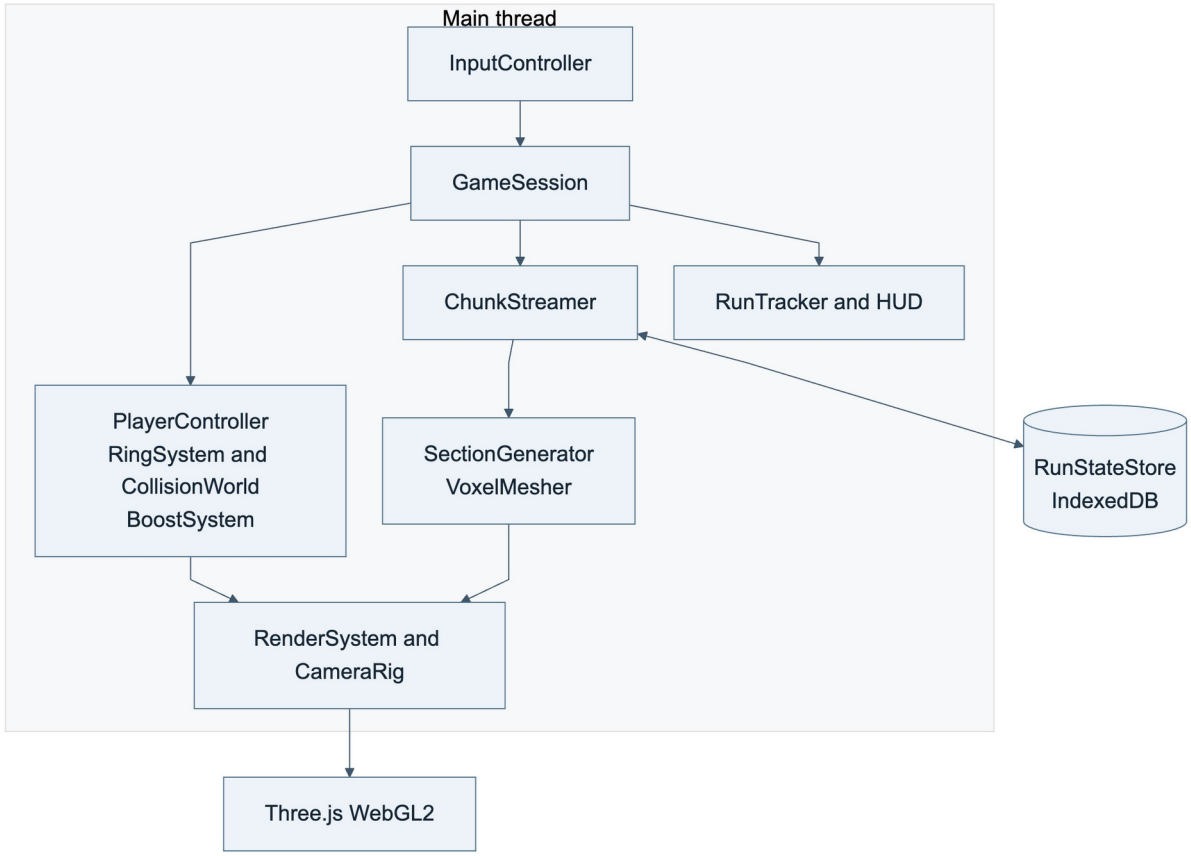
Keep world presentation consistent across items and pads: dash uses a forward-chevron icon, high jump an upward arrow, and glide a wing icon. Candidate accents are dash `#F3AF49`, high jump `#77DA94`, and glide `#80CAF3`. The HUD shows both slot keys, held icons, and active-effect progress. Successful activation gives brief visual/audio feedback; rejected activation leaves the slot unchanged and gives a subtle unavailable cue. Speed effects must preserve platform edges.

Milestone 1 establishes baseline jump height, airtime, wall movement, and ring interaction; Milestone 3 tunes boost duration/speed and placement against those measurements. Record each change in the movement-rules version. Validate duplicates, full inventory, airborne high jump, combined glide/dash, maximum-speed collision, and pad reentry before enabling procedural placements. Roadmap and Validation defines the future tests; this catalogue does not report completed gameplay verification.

Proposed design diagrams

01 Single-thread custom 2.5D

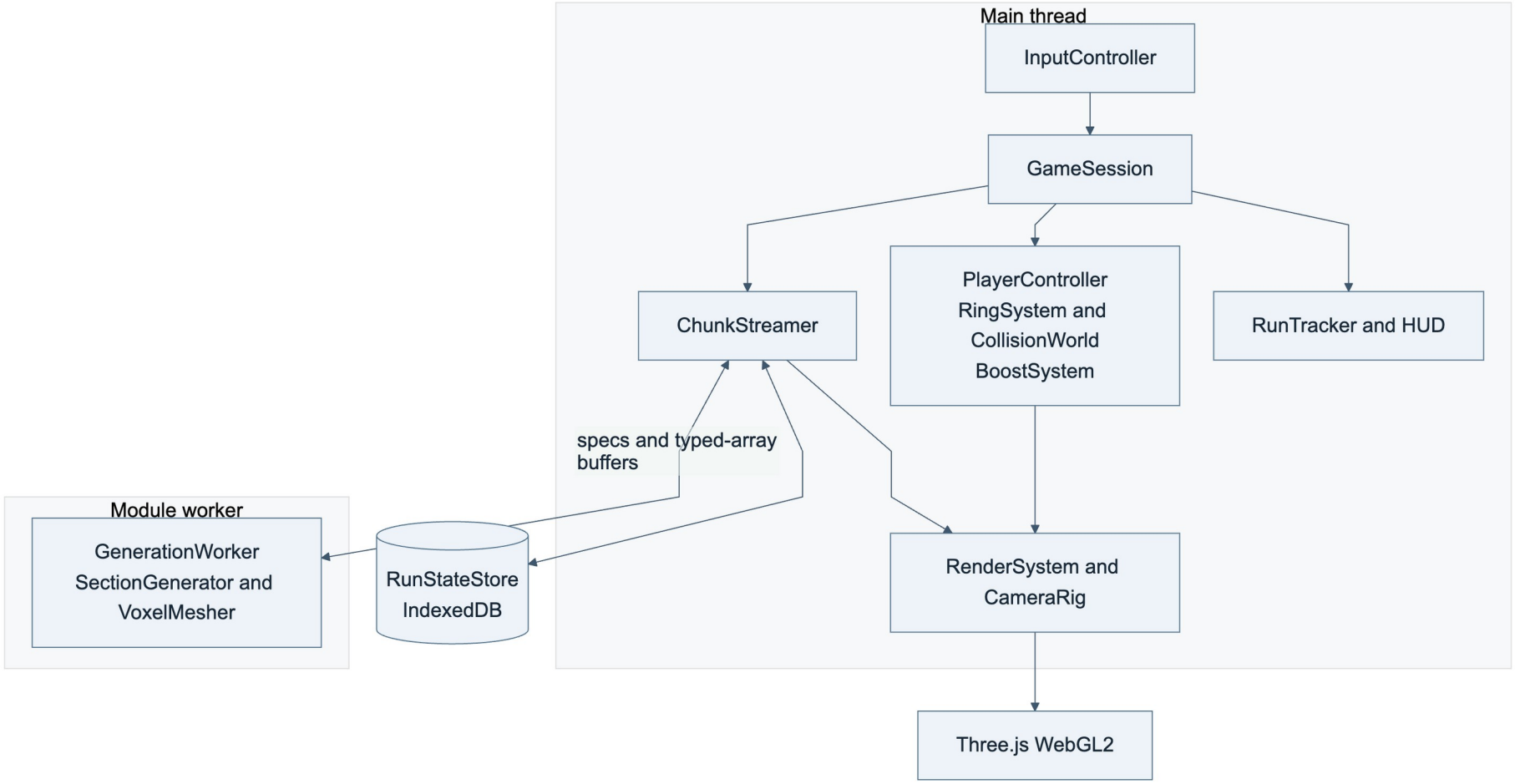
These diagrams describe the planned game, not implemented code. Four compact class views together form the Design Class Diagram. Types and system names match the architecture contracts. Editable Mermaid sources accompany each rendered image. Multiplicities show intended instance counts; dotted arrows denote dependencies. All gameplay, generation, meshing, and rendering share the main thread. This is the smallest prototype but exposes movement to generation stalls. Mermaid source



Editable source: 01-architecture-single-thread.mmd

02 Modular 2.5D with generation worker

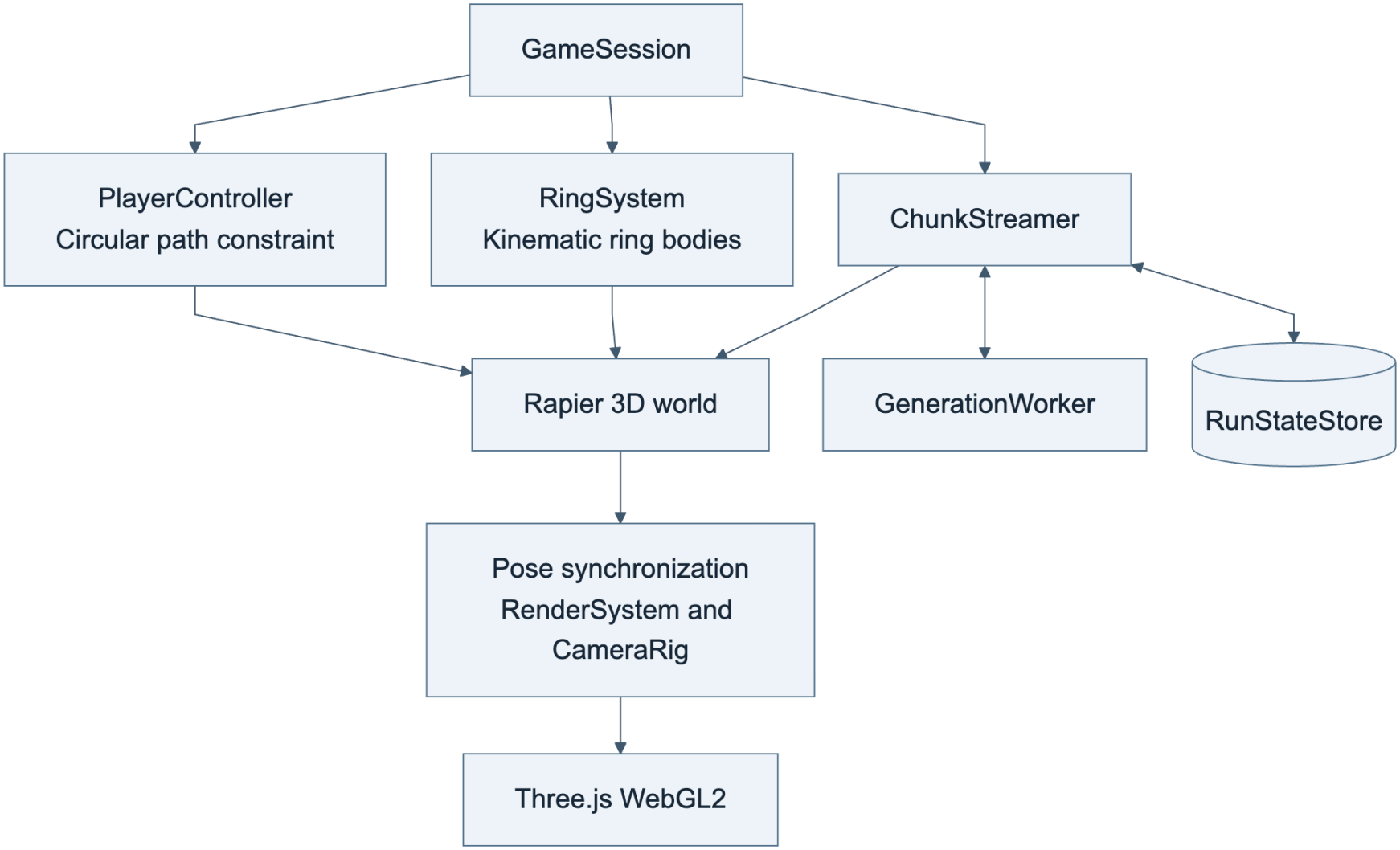
Recommended architecture: gameplay remains on the main thread while a module worker builds sections and transferable mesh buffers. Mermaid source



Editable source: 02-architecture-worker.mmd

03 Constrained Rapier 3D alternative

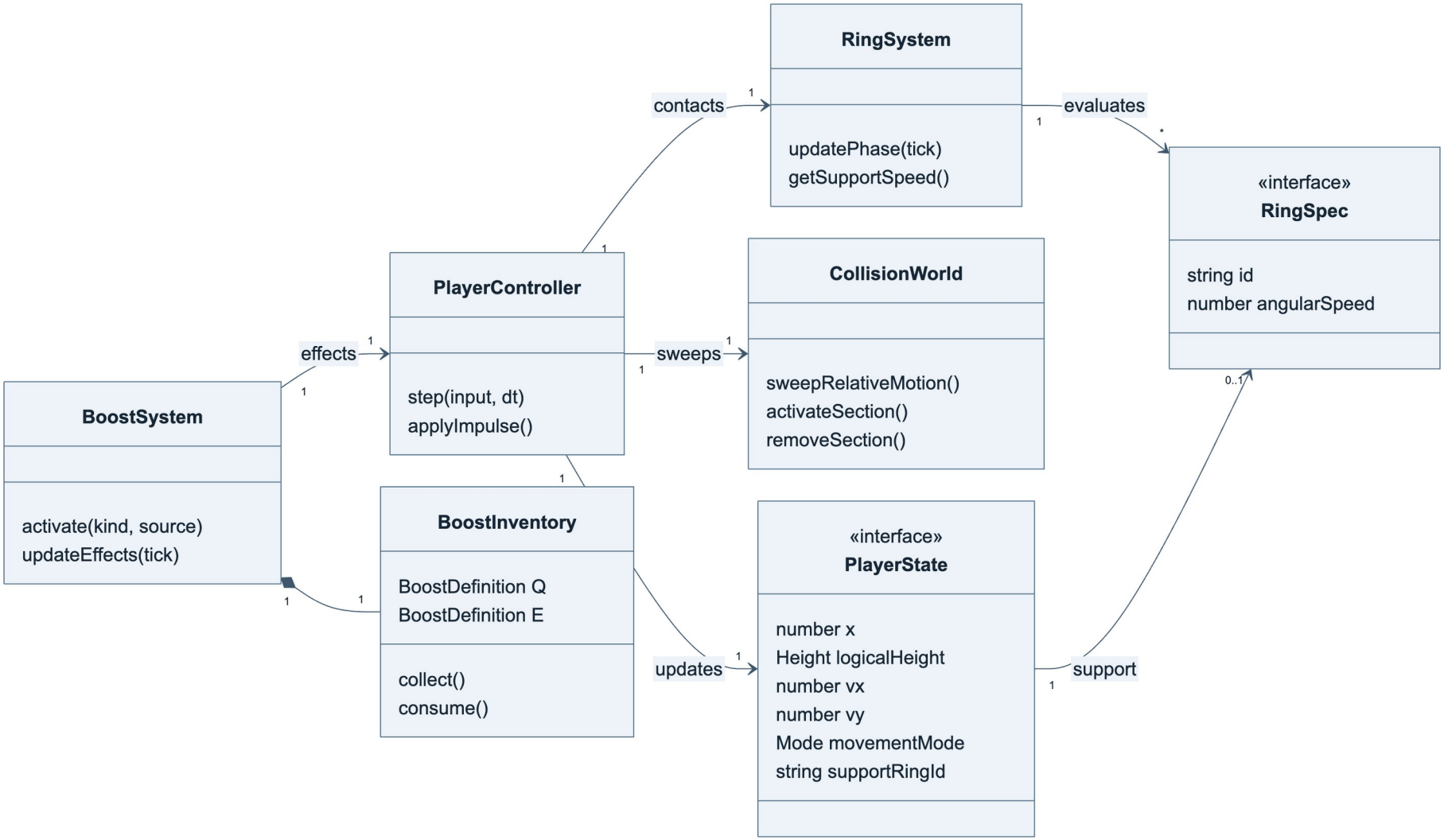
A 3D physics world adds circular constraints and rotating-body synchronization. Reserve this alternative for a design that needs radial movement. Mermaid source



Editable source: 03-architecture-rapier.mmd

04 Movement and boosts

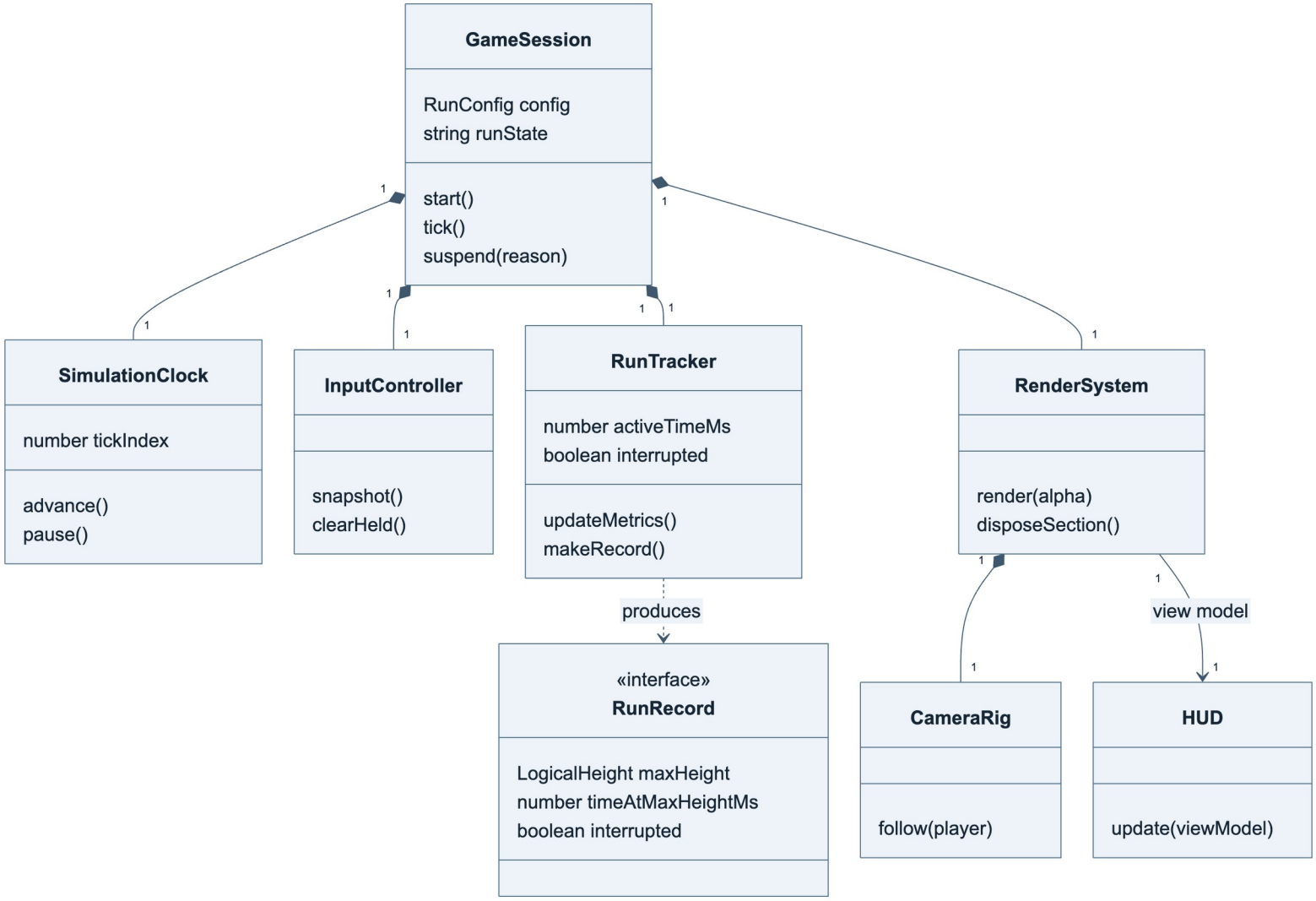
Player state, relative collision, ring contact, and inventory form the movement view. Support is optional; boosts remain separate from movement mode. Compact labels use `Height` for `LogicalHeight` and `Mode` for `MovementMode`. Mermaid source



Editable source: 04-dcd-runtime.mmd

12 Session and presentation

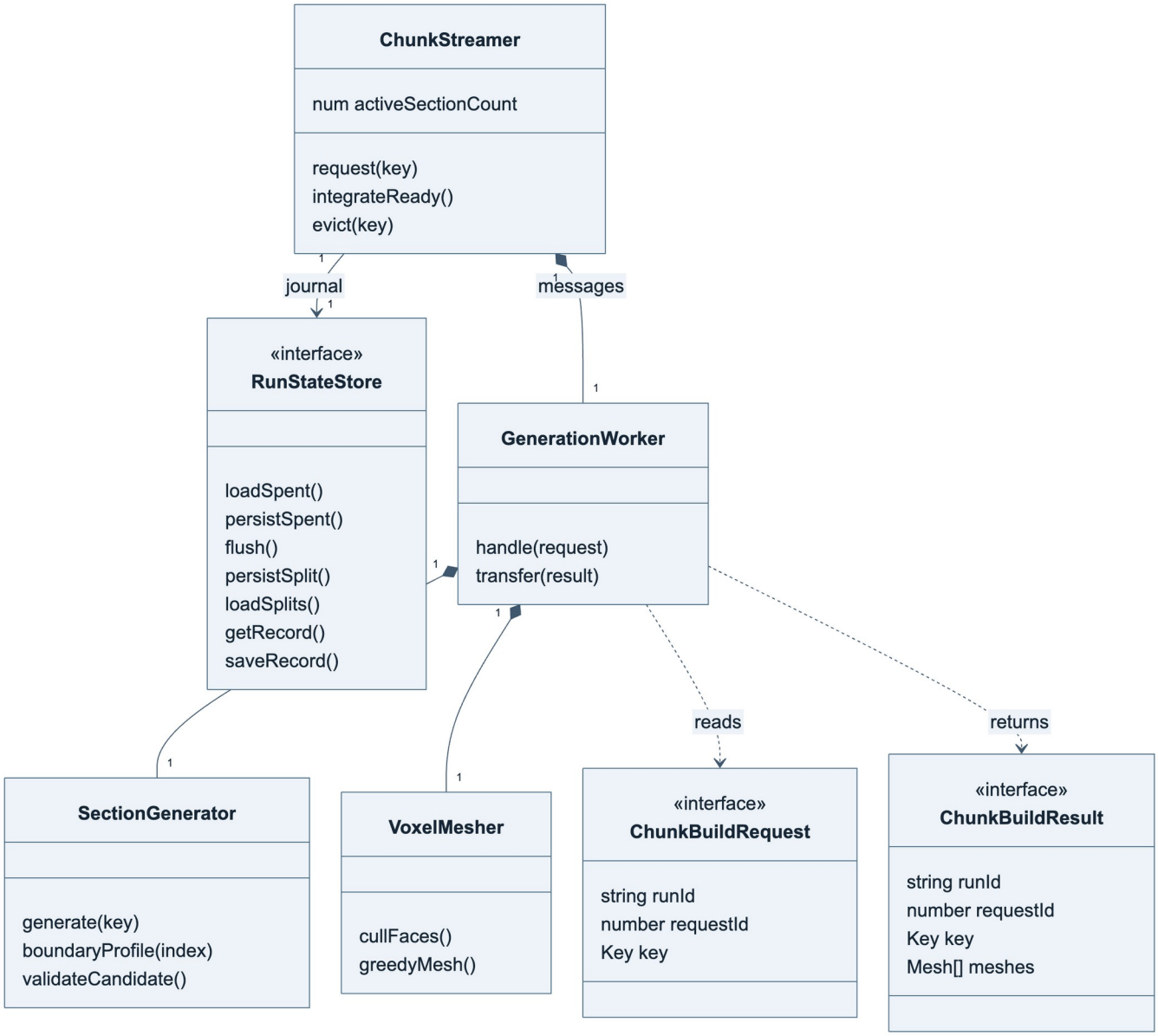
The session owns input, timing, presentation, and metrics. Rendering follows authoritative state without making movement decisions. Mermaid source



Editable source: 12-dcd-session-presentation.mmd

05 Worker pipeline and persistence

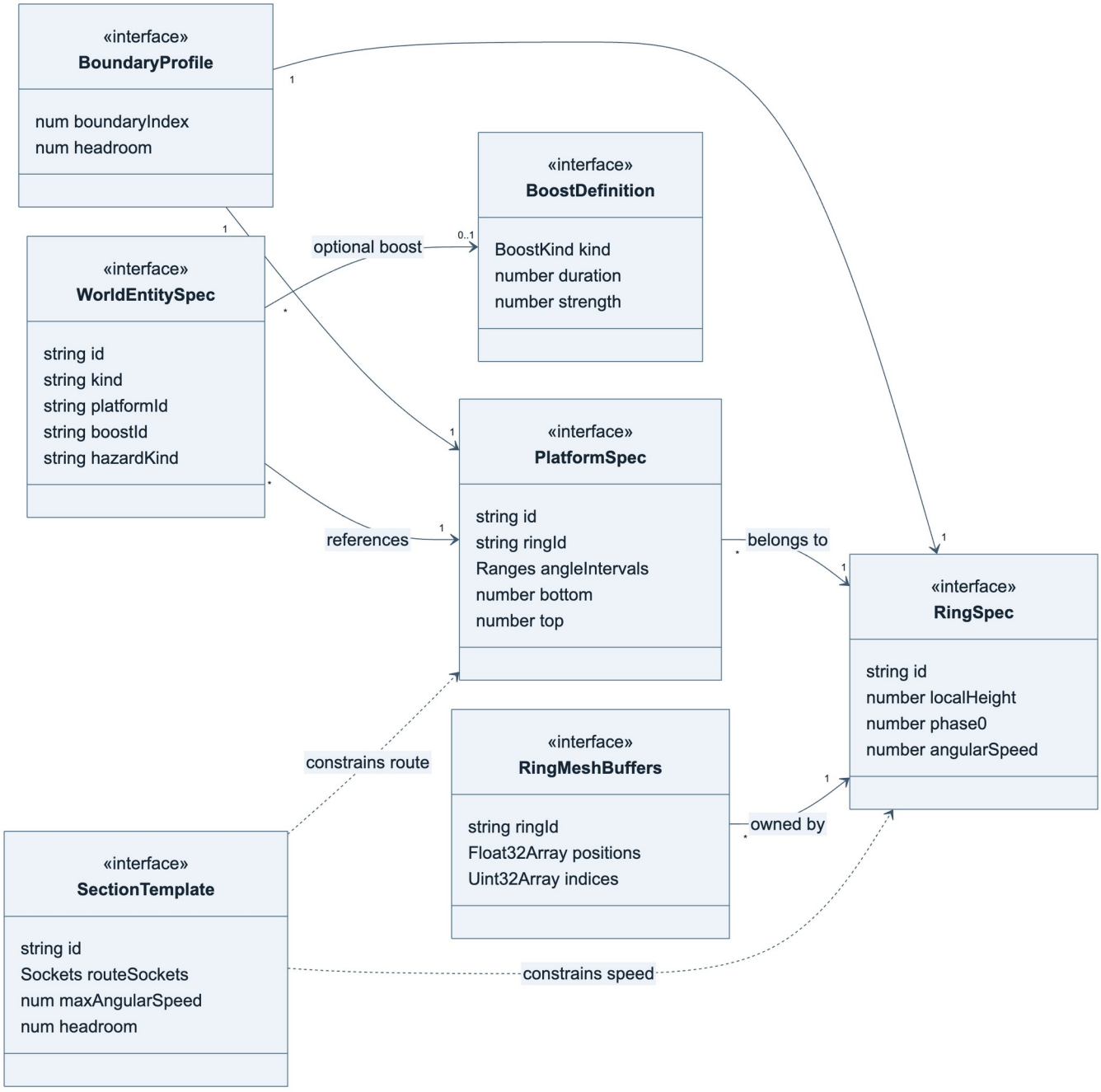
The streamer combines worker output and the journal before activation. Requests/results contain plain data and transferable buffers. Compact labels use `num` for `number`, `Key` for `SectionKey`, and `Mesh` for `RingMeshBuffers`. Mermaid source



Editable source: 05-dcd-world-pipeline.mmd

13 World data model

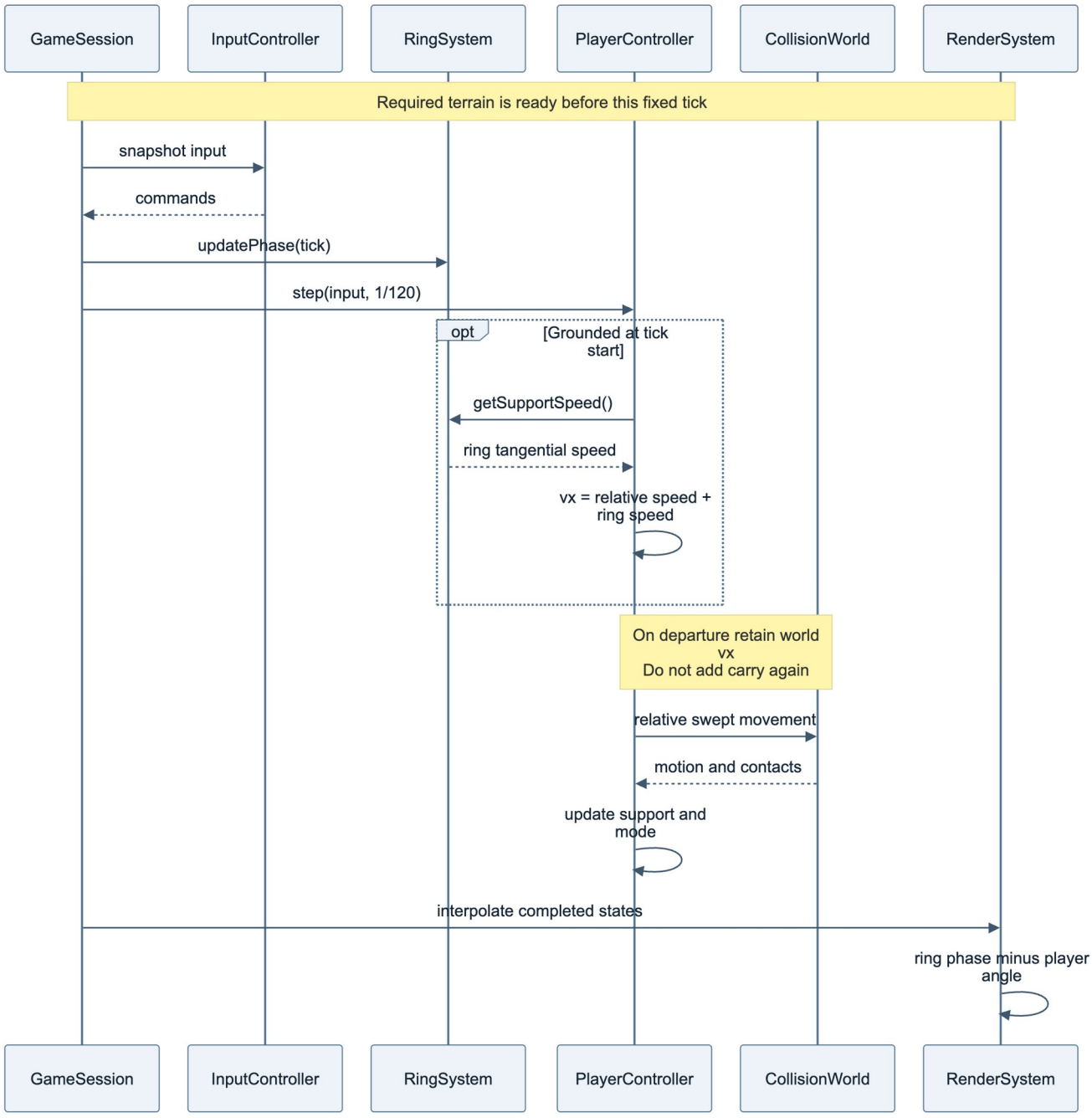
Platforms and entities reference rings and shared effects. Canonical boundaries coordinate neighbours; mesh buffers retain ring ownership. Compact labels use `num` for `number`, `Sockets` for `RouteSocket[]`, `Ranges` for the platform's angle-interval array, and `headroom` for `reservedHeadroom`. Full types are in the architecture contracts. Mermaid source



Editable source: 13-dcd-world-data.mmd

06 Movement and rotating contact

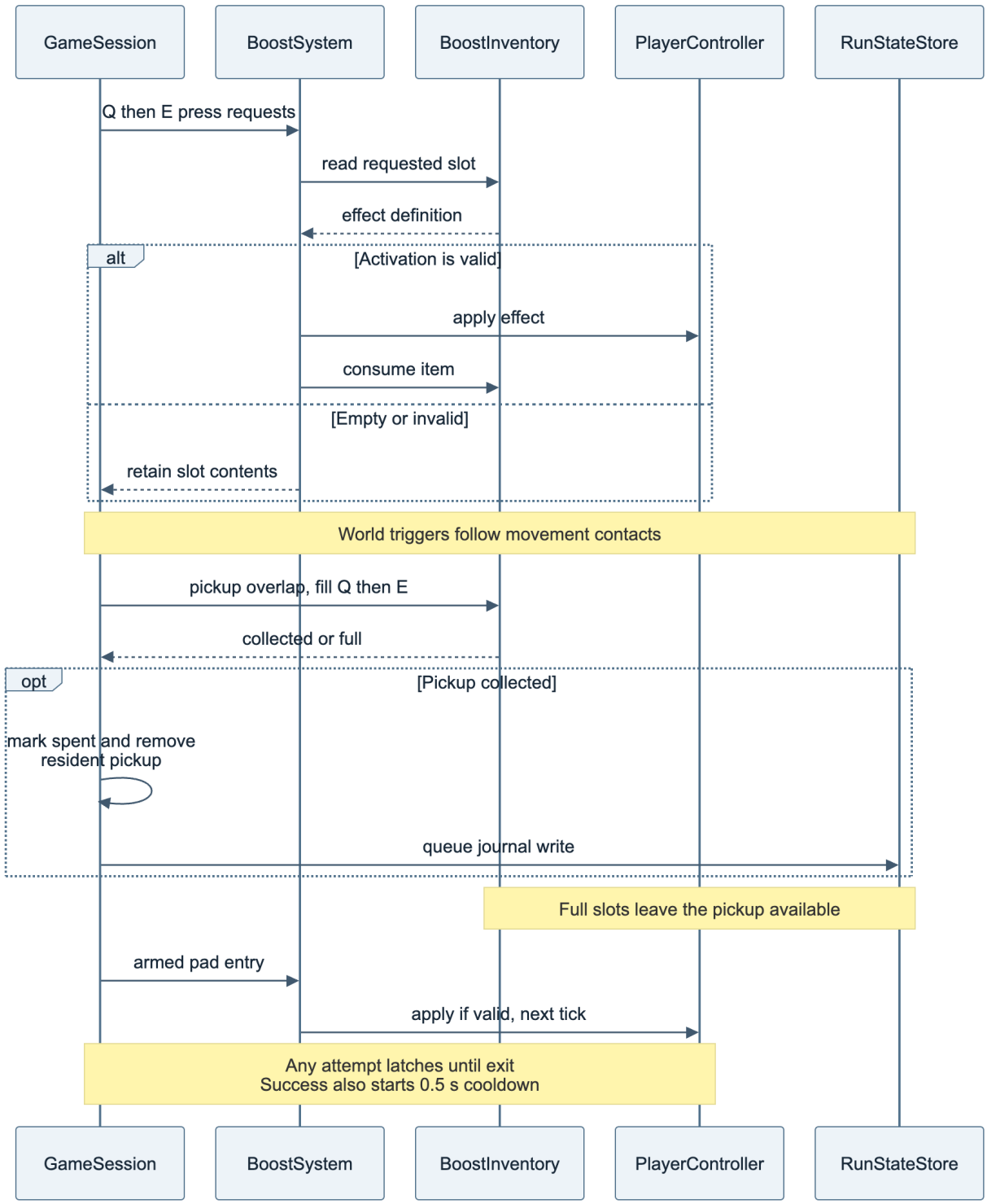
Ring phase precedes the movement sweep. Contact carries grounded players, while takeoff inherits ring speed once.
Mermaid source



Editable source: 06-sequence-movement.mmd

07 Items and environmental boosts

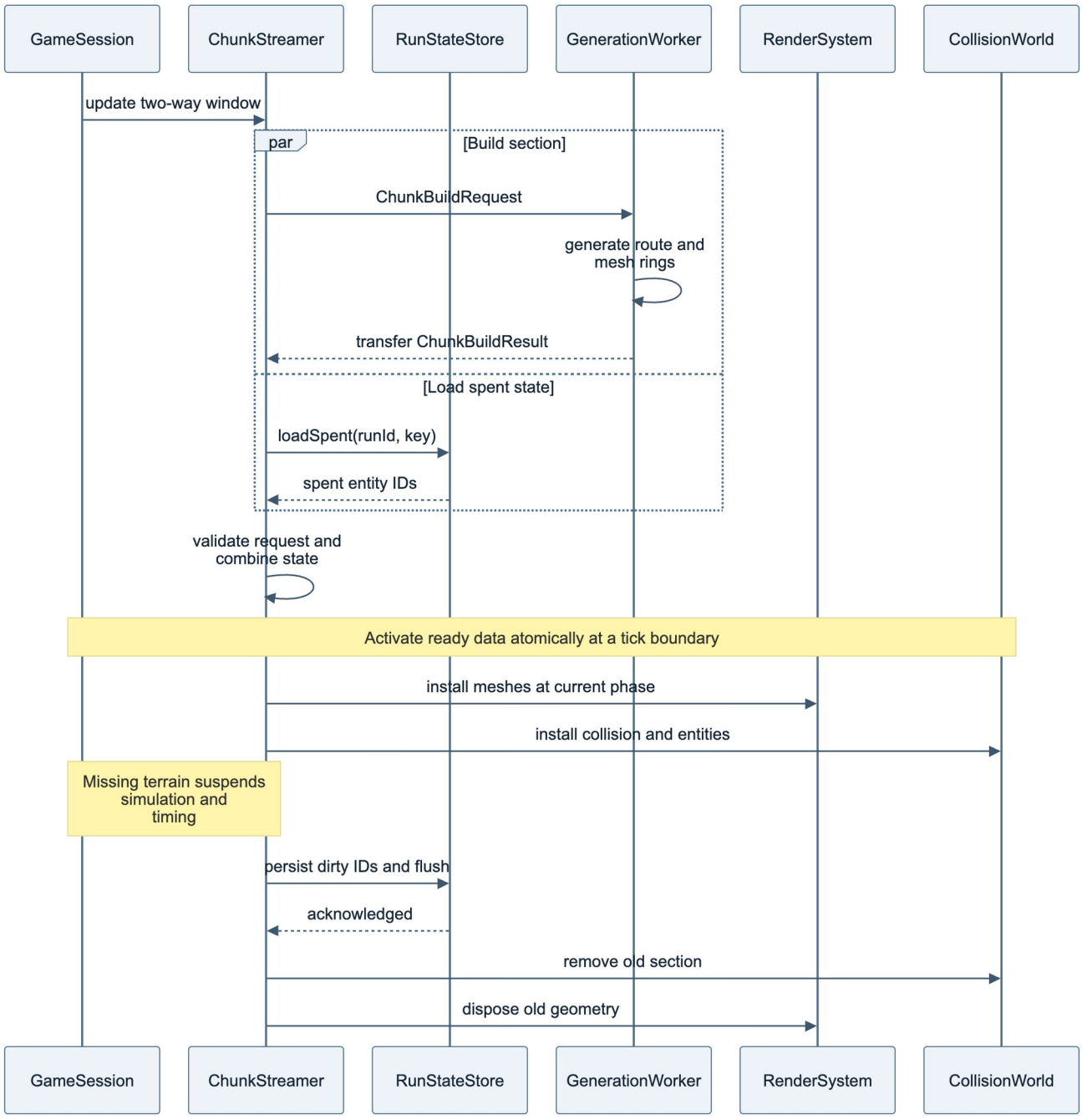
Inventory items and pads share effect definitions. Successful activation consumes items; pads rearm after exit and cooldown. Mermaid source



Editable source: 07-sequence-boost.mmd

08 Bidirectional streaming

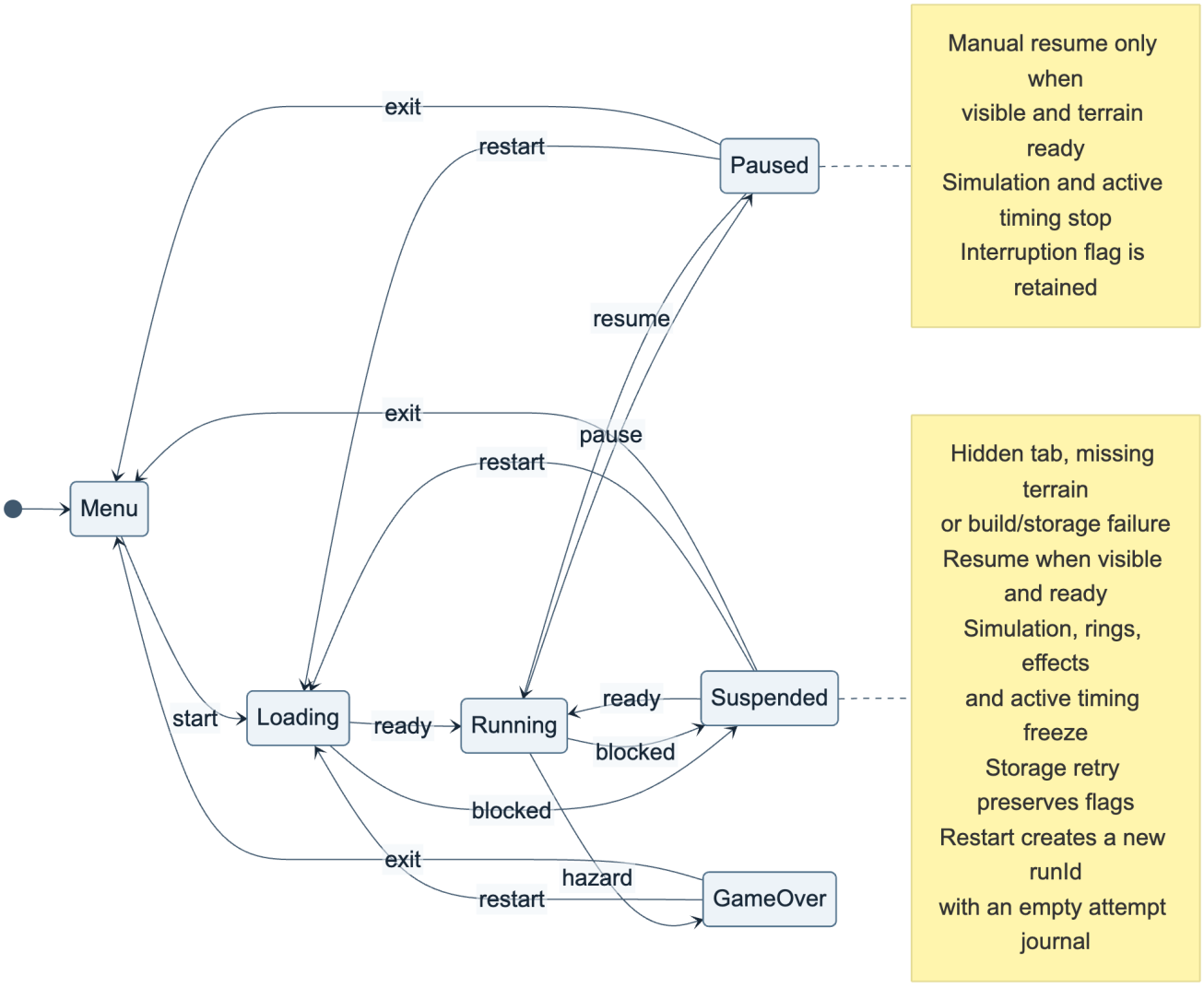
Spent-state loading and worker generation converge before atomic activation. Eviction waits for persistence. Mermaid source



Editable source: 08-sequence-streaming.mmd

09 Run lifecycle

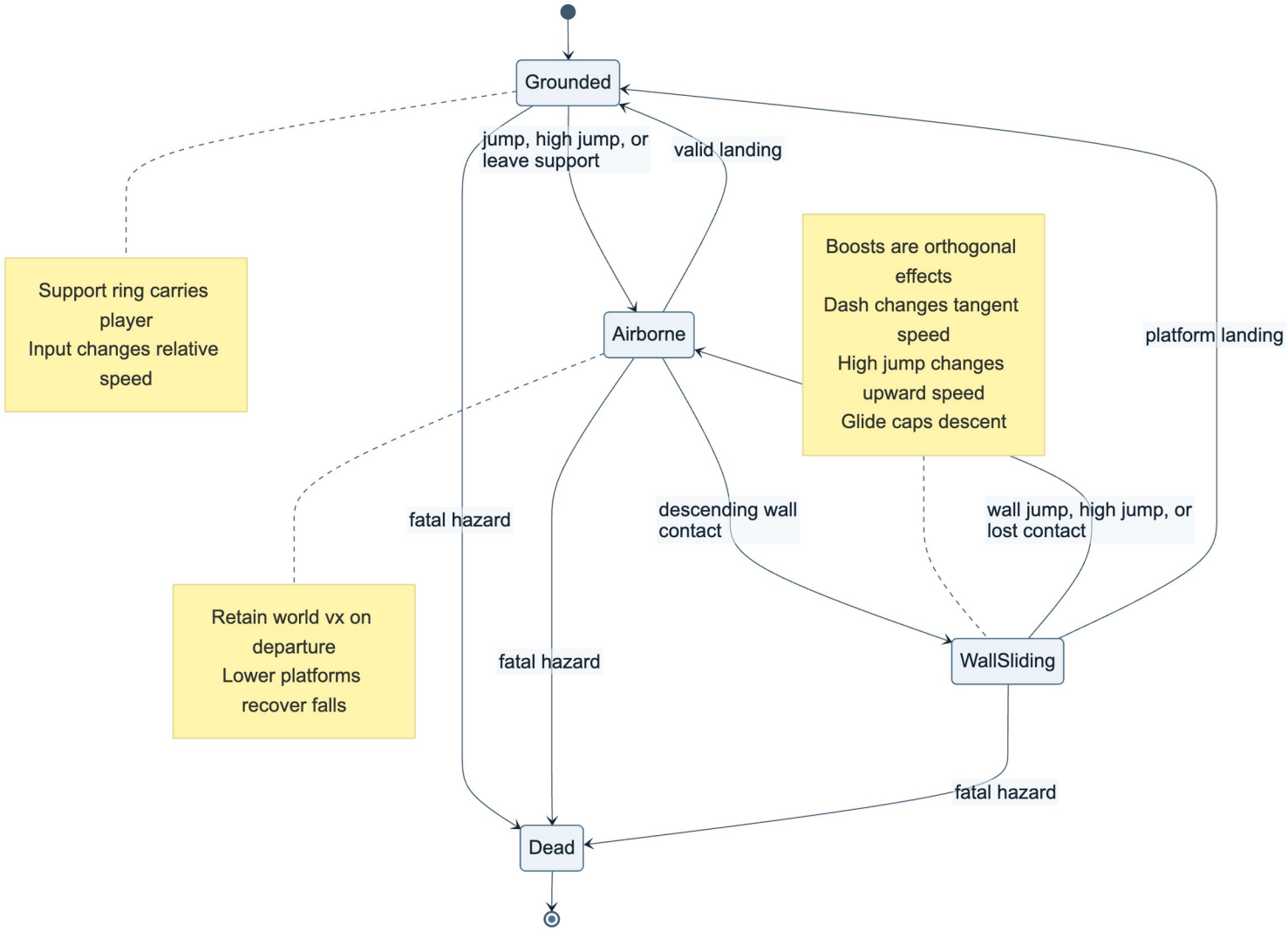
Suspensions stop gameplay and active timing together. Restart creates a new attempt; it does not resume at a checkpoint.
Mermaid source



Editable source: 09-state-run.mmd

10 Movement state and boost overlay

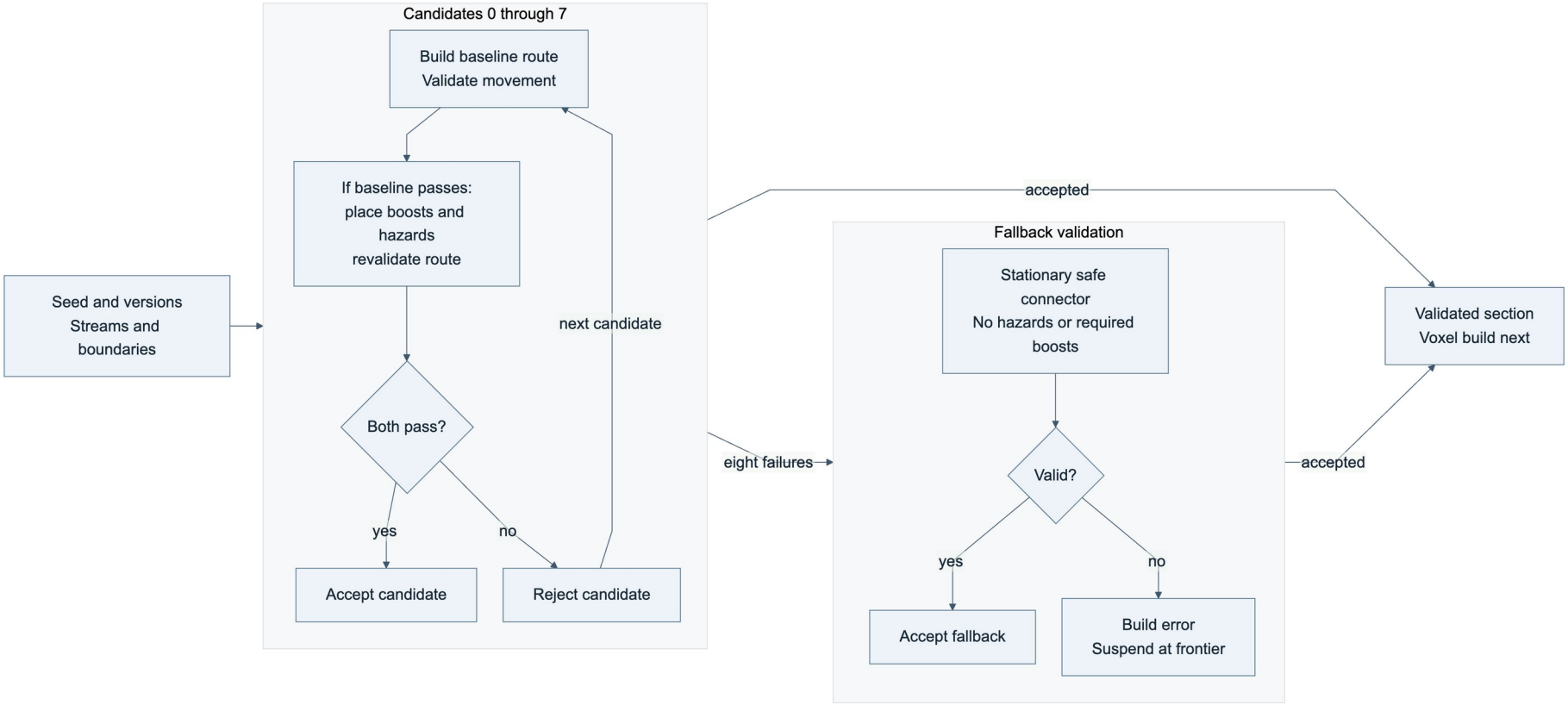
Grounding, airtime, wall slide, and death are movement modes. Boost effects modify those modes without replacing them. Mermaid source



Editable source: 10-state-movement.mmd

11 Seeded route generation

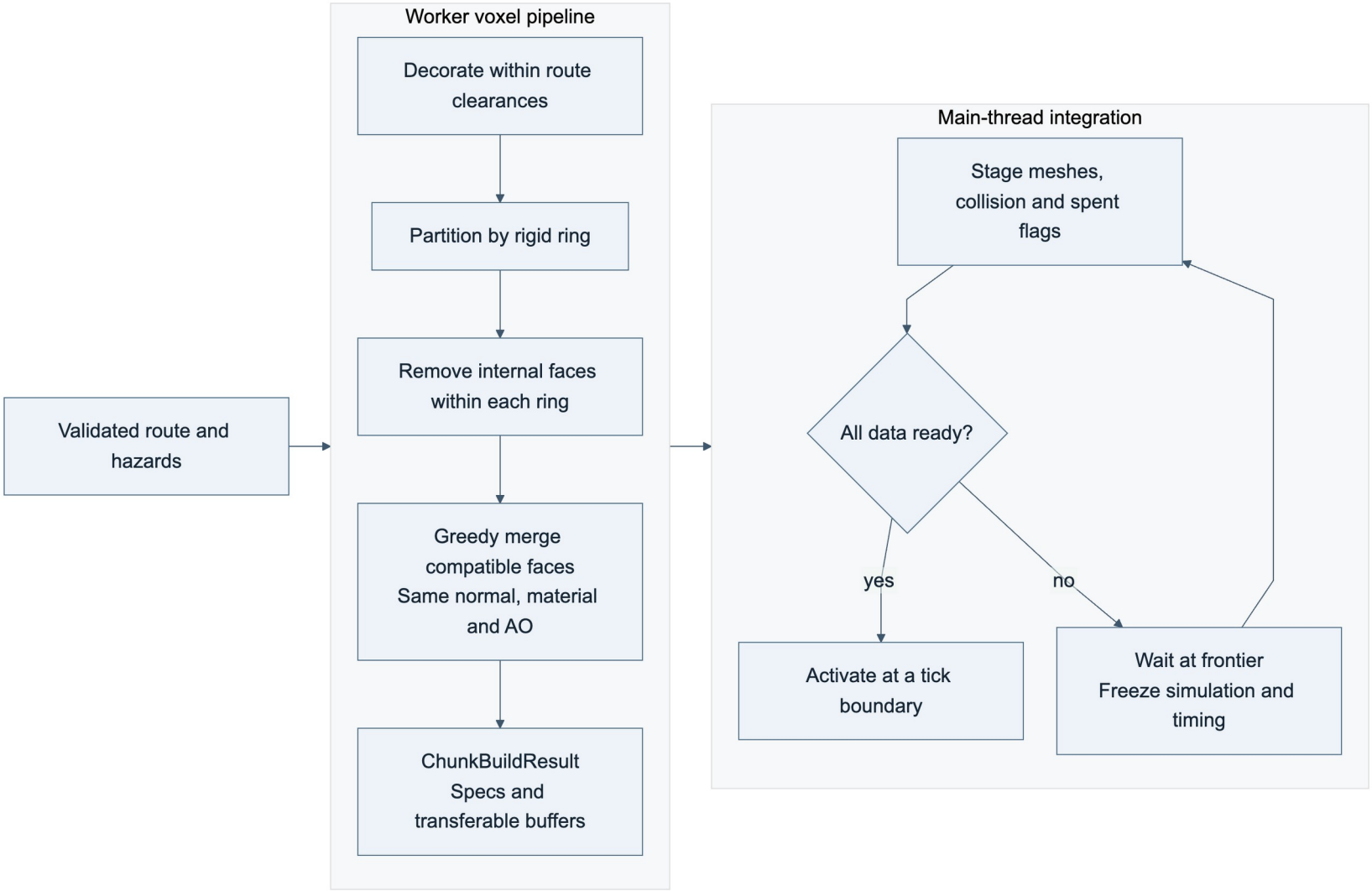
Canonical boundaries precede candidate validation and voxel decoration. A fixed retry limit leads to a safe deterministic fallback. Mermaid source



Editable source: 11-flow-generation.mmd

14 Voxel build and activation

After route validation, decoration preserves gameplay clearances. Each rigid ring is meshed separately, and the streamer activates visual and collision data with the spent-item flags. Mermaid source



Editable source: 14-flow-voxel-build.mmd

Implementation Roadmap and Validation

Delivery sequence

The six milestones retain the proposal's ordering. Each milestone has an observable exit condition so the team can resolve movement and streaming risks before investing in graphical polish. Dates remain unset. The ownership below is a proposed division based on the proposal, with integration shared across the team.

Milestone	Work and owner	Exit condition
M1 Baseline engine	Omar: Three.js scene and camera. Alexandru: input, cylindrical movement and rotating-ring collision.	A small authored course demonstrates centered wraparound movement, ring carry, inherited jump momentum, wall movement and swept collision at candidate maximum speeds.
M2 Deterministic world	Adrian: keyed section generator, boundary profiles and templates. Omar: exposed-face extraction and greedy meshing.	Repeated generation produces identical gameplay data regardless of request order. Route validation rejects invalid candidates and selects a safe fallback.
M3 Streaming and mechanics	Adrian: bidirectional streamer and run journal. Alexandru: inventory, boost effects, pads and hazards.	Ascending and descending through evicted sections preserves collected-item state and ring phases. Meshes, collision and journal flags activate together.
M4 Lighting and atmosphere	Alexandru: GLSL lighting, voxel ambient occlusion, shadows and parallax. Omar: rendering integration.	All six environment treatments preserve platform and hazard readability. Quality controls demonstrate the cost of individual passes.
M5 Speedrunning systems	Alexandru: timer, section splits, seed entry, records and speed effects. Shared: HUD integration.	Same-course local records use consistent timing flags. Falls preserve height records and first-arrival splits. Restart resets run state.
M6 Optimization and polish	All: performance profiling, playtesting, tuning, resource disposal and course report.	The team publishes measurements, settings, hardware, failures and limitations for the benchmark protocol below.

The first integration gate is M1. If rotating contacts or wall jumps remain unreliable, simplify the authored course and repair the controller before procedural variety. At M2, implement Stone Ruins and Volcanic Forge first to establish ordinary traversal and a lethal hazard. The remaining environment definitions already exist in the catalogue and join the same generator contracts.

Gameplay acceptance scenarios

These are specifications for future tests. No game executable or gameplay test results accompany this preproduction package. Use Vitest for generation and controller scenarios that can run without rendering, and Playwright plus human playtesting for browser presentation and complete runs.

ID	Scenario	Observable acceptance
TC-01	Controls and camera	A/D and arrows wrap around the tower, Space jumps, and the avatar remains centered without radial input or camera discontinuities.
TC-02	Ring carry	With no input, a supported player follows the ring. Input moves relative to it. Leaving the support interval removes support.
TC-03	Jump momentum and opposing landing	Takeoff inherits support velocity once. Airborne motion never repeatedly adds it. Landing on an opposing ring changes the support frame without teleporting.

ID	Scenario	Observable acceptance
TC-04	Wall movement	Wall contact caps descent; a buffered jump pushes away and upward. Jumping into ceilings or corners remains collision-safe.
TC-05	Wraparound and sweeps	Equivalent intervals on either side of the circumference seam collide identically. Maximum permitted velocities cannot skip thin walls or landing surfaces.
TC-06	Dash	A valid activation applies the configured tangential burst and respects collision. A second active dash rejects without spending a charge.
TC-07	High jump	Activation raises upward velocity to the configured launch value from ground, wall slide or air. Velocity already at or above that value rejects without spending.
TC-08	Glide and combinations	Glide caps descent without reducing ascent and ends on landing or expiry. Dash and high jump can coexist with glide without multiplying its strength.
TC-09	Inventory	Pickups fill Q then E; duplicates work; full slots leave items untouched. Held keys do not repeat. Simultaneous Q/E uses a stable order and spends only successful activations.
TC-10	Pad rearming	Entry triggers once. Continuous overlap never repeats. Full exit plus the cooldown rearms the pad. Invalid activation cannot create a per-tick activation loop.
TC-11	Falling, death and retry	Ordinary descent lands on available lower geometry or the starting base. Only designated lethal hazards end the run. Retry restores the seed's initial world and clears inventory/history.
TC-12	Seed and ordering	Identical course identity produces byte-identical canonical gameplay descriptions for repeated, ascending, descending and shuffled section requests. Changing a version isolates records.
TC-13	Reachability and fallback	Baseline movement can complete accepted transitions across sampled phases without held items. Exhausted attempts validate the deterministic safe template; failed fallback validation returns an error and safely suspends before entry.
TC-14	Themes and hazards	All six themes keep platforms, walls, pickups, pads and lethal hazards recognizable. Decorative frost and wind do not secretly alter movement.
TC-15	Descent and spent items	Collect, unload, descend and revisit a section. Collected items remain absent, uncollected items remain available, and ring phase matches current run time.
TC-16	Worker and activation	Delayed or stale responses from an old run never activate. A section becomes traversable only when visuals, collision, triggers and spent flags are ready together.
TC-17	Capabilities, suspension and timing	Missing WebGL2, worker or required storage capability produces useful startup feedback. Pause, hidden tab, missing terrain and storage failure freeze simulation, rings, effects and active elapsed time. Resuming causes no catch-up jump. Post-start interruptions flag the record.
TC-18	Height, splits and records	First upward crossing writes each split once. Descent does not erase it or the maximum height. Local comparisons use matching course identity and timing classification.
TC-19	Origin rebasing	Crossing each rebase threshold preserves contact, logical height, camera, ring phase and trigger identity. Repeat during ascending and descending movement.
TC-20	Resource and performance soak	Complete the protocol below with active resources within caps, no missing terrain, and no upward trend in live section resources after repeated eviction.

Generation validation

Start with 100 canonical seeds, sections 0 through 99, and 16 rotation-phase samples per applicable transition. Include seed 0 and the maximum uint32 seed, maximum difficulty, template boundaries and every environment. Run identical batches with shuffled section requests and different worker completion schedules.

Use the actual controller as the final transition oracle. Analytical jump bounds are preliminary filters. Save a failure's seed, versions, section, candidate number, tick and controller state so the team can reproduce it. Template-level sampling and playtests provide evidence, not a mathematical proof that every possible run is traversable.

Performance benchmark protocol

Record machine model, CPU, GPU, RAM, operating system, browser version, dependency versions and selected graphics quality. The initial reference setting is a 1920 by 1080 drawing buffer, device pixel ratio 1, foreground tab and a 60 Hz display. Use the same settings and seed set for comparisons.

After a two-minute shader and streaming warmup, run a 30-minute traversal workload that ascends, falls, revisits sections and uses maximum-speed boosts. Record frame-time percentiles, main-thread generation work, worker build time, upload/integration time, visible loading interruptions, live geometry/texture counts, draw calls, triangles, estimated buffer bytes, active sections and pending queues. Report ordinary traversal separately from frontier suspensions; do not hide stalled time by omitting it from measurements.

Measure	Initial target or bound	Interpretation
Frame delivery	60 FPS target; p95 frame interval at most 16.7 ms and p99 at most 33.3 ms	Candidate acceptance targets on the documented reference machine, not measured results or a cross-device guarantee.
Worker section build	p95 at most 150 ms	Candidate budget used when sizing prefetch; tune from measured output complexity.
Resident sections	Six normally; eight maximum	Includes recovery terrain and adaptive prefetch.
Worker and ready queues	Two in flight; two completed results	Apply backpressure and drop obsolete work.
Streaming correctness	Zero missing-terrain collisions or deaths	An unavailable frontier causes visible suspension.
Resource lifetime	No sustained growth in live section geometry or collision buffers after eviction	Persistent journal disk growth is reported separately.

Three.js exposes draw calls, primitive counts and live geometry/texture counts through `renderer.info`; those counts are not a direct GPU byte measurement. Estimate owned buffer sizes and label estimates. Explicitly dispose evicted geometry and release shared resources only when their reference count reaches zero. Renderer statistics, resource disposal.

Risk register

Risk	Mitigation and gate
Rotating contacts produce unfair jumps	Test carry, seams, wall contact and opposing landings on an authored M1 course before generation.
Procedural sections strand the player	Shared boundary profiles, safe waiting surfaces, baseline routes, controller validation and bounded retries with fallback at M2.
Boosts outrun generation	Velocity caps, directional prefetch, bounded jobs and a visible ready-frontier suspension at M3.
Revisits restore spent pickups	Stable entity IDs, journal flags applied before activation, persistence before eviction and storage-failure suspension.
Materials or ring transforms break meshing	Mesh per rigid ring, merge only compatible faces, and keep collision tied to shared platform descriptions.
Lighting obscures gameplay or exhausts frame budget	Readability checks in each theme and individually switchable shadow/post-processing passes at M4.

Risk	Mitigation and gate
Timing comparisons become misleading	Active monotonic clock, explicit interruption classification, course/rules version separation and no input-replay claim.

Planning-package validation

AC-01 checks required documents and contiguous FR, NFR and TC identifiers. AC-02 checks that requirements reference existing test IDs and that every scenario has at least one requirement. AC-03 parses all Mermaid diagrams and verifies their source/image references. AC-04 checks local Markdown links and required interface/theme names. AC-05 renders the DOCX and PDF and inspects every page for readable tables, diagrams and code, with no clipping or overlap. AC-06 compares the exported report's content and figure inventory with the Markdown manifest. Package verification is recorded separately from future gameplay acceptance.

Sources and Design Decisions

Project basis

The project proposal is Computer Graphics Project Proposal - Group 7, originally titled SPEEDFORM, for Group 7 in CS 4361.002. It names Adrian Alvarez, Alexandru Istrate and Omar Sabry. An unchanged copy accompanies this package. The current project name is SpeedTower.

Proposal location	Design input
Slides 1 and 3	Endless tower climbing, repeatable seed-based layouts and speedrunning motivation.
Slide 4	Voxel meshing, hidden-face removal, streamed world construction and custom lighting.
Slide 5	Movement, timer, seed input, rendering/generation responsibilities, profiling, ambient occlusion and post-processing.
Slide 6	Six ordered implementation milestones.
Slide 7	Playable platformer, shader suite, seed sharing and performance report.

The selected gameplay specification adds circumferential side-view movement, independently rotating rings, Q/E item slots, matching immediate boost pads, wall jumping, recoverable falls, no checkpoints and six themed environment definitions. The chosen initial boost effects are dash, high jump and glide.

The design interprets infinity as on-demand generation with bounded resident world resources. One draw call, instantaneous buffer replacement and strictly constant overhead are not guaranteed. Course reproducibility also does not establish cross-browser deterministic input replay.

Architecture decision register

ID	Decision	Reason
ADR-01	Direct Three.js and a custom cylindrical 2.5D controller	The fixed radial track reduces movement to two logical axes while retaining 3D graphics.
ADR-02	Main-thread simulation with worker-owned generation and meshing	Keeps collision and input straightforward while separating expensive world construction.
ADR-03	Route-first templates with shared boundaries	Arbitrary density noise cannot establish reliable platforming routes.
ADR-04	Versioned seeded course identity	Separates reproducible maps and comparable records after generator or movement changes.
ADR-05	Bidirectional streaming with a spent-item journal	Allows recoverable falls without retaining all geometry or restoring collected items.
ADR-06	Local records with interruption classification	Supports seed-based practice and comparison without adding an online service.
ADR-07	Shared boost definitions with separate item and pad triggers	Both acquisition modes produce the same movement effect without mixing inventory and world-trigger state.

Technical references

References were checked during preproduction on October 6, 2026. Pin implementation dependency versions when the game scaffold is created, and update the course versions when relevant behavior changes.

- Three.js WebGLRenderer: WebGL2 renderer, frame statistics, compilation and rendering controls.
- Three.js voxel geometry: exposed-face geometry and division into smaller voxel cells.
- Three.js object disposal: resource lifetime and explicit GPU disposal.
- Three.js InstancedMesh: repeated prop rendering; not a substitute for removing internal voxel faces.
- Vite guide: TypeScript-capable development and static production builds.
- MDN Web Workers: isolated background work and messaging.
- MDN transferable objects: ownership transfer of array buffers.
- Rapier character controller: physics alternative and controller rotation limitations.
- Rapier determinism: conditions required for deterministic physics, distinct from seeded course generation.
- Mermaid class diagrams: editable UML-style class notation, relationships and multiplicities.

Status of evidence

Gameplay tuning, reachability samples and performance budgets in this package are proposed engineering targets. M1 movement experiments, generation tests, playtests and the M6 benchmark must establish their results. Document rendering and link/traceability checks validate the planning artifacts only.